WEBOVÁ APLIKACE PRO SPRÁVU TANEČNÍCH AKCÍ A KURZŮ

Bc. Žaneta Trošková

Diplomová práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Webové inženýrství
Vedoucí: Ing. Milan Dojčinovski, Ph.D.
30. března 2026

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ ČVUT V PRAZE

Zadání diplomové práce

Název:	Webová aplikace pro správu tanečních akcí a kurzů
Student:	Bc. Žaneta Trošková
Vedoucí:	Ing. Milan Dojčinovski, Ph.D.
Studijní program:	Informatika
Obor / specializace:	Webové inženýrství
Katedra:	Katedra softwarového inženýrství
Platnost zadání:	do konce letního semestru 2026/2027

Pokyny pro vypracování

Společenské tance (social dance), jako jsou například bachata, salsa a zouk, jsou tance provozované hlavně pro radost, setkávání a zábavu, nikoli pro soutěže. Organizování těchto akcí je spojeno s řadou organizačních úkonů, které jsou často realizovány prostřednictvím různých nesjednocených nástrojů, což ztěžuje pořadatelům jejich správu i uživatelům přehledné vyhledávání a přihlašování na akce.

Cílem diplomové práce je návrh a implementace webové aplikace, která usnadní pořadatelům social dancing akcí a kurzů, především v oblasti bachata, salsa a zouk, jejich správu. Zároveň aplikace umožní uživatelům přehledné vyhledávání akcí a jednoduché přihlašování k účasti.

Úkoly práce:
- Analyzovat požadavky pořadatelů a účastníků social dancing akcí a kurzů se zaměřením na tance bachata, salsa a zouk.
- Na základě provedené analýzy navrhnout architekturu a funkcionalitu webové aplikace.
- Vytvořit návrh uživatelského rozhraní aplikace.
- Implementovat navrženou webovou aplikaci.
- Provést vhodné testování vytvořeného řešení.
- Zhodnotit dosažené výsledky a navrhnout možná budoucí rozšíření a vylepšení aplikace.

Elektronicky schválil/a Ing. Jaroslav Kuchař, Ph.D. dne 9. února 2026 v Praze.

České vysoké učení technické v Praze
Fakulta informačních technologií
© 2026 Bc. Žaneta Trošková. Všechna práva vyhrazena.
Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Trošková Žaneta. *Webová aplikace pro správu tanečních akcí a kurzů.* Diplomová práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

Ráda bych poděkovala především svému vedoucímu, Ing. Milanu Dojčinovskému, Ph.D., za jeho cenné rady, vstřícnost, podporu a trpělivost během celého procesu tvorby této diplomové práce. Dále děkuji členům taneční komunity, kteří se mnou sdíleli své zkušenosti, poskytli mi užitečnou zpětnou vazbu a zapojili se do uživatelského testování. V neposlední řadě patří mé poděkování rodině a přátelům za jejich podporu a oporu po celou dobu studia i psaní této práce.

Prohlášení

Prohlašuji, že jsem předloženou práci vypracoval samostatně a že jsem uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů. V souladu s ust. § 2373 odst. 2 zákona č. 89/2012 Sb., občanský zákoník, ve znění pozdějších předpisů, tímto uděluji nevýhradní oprávnění (licenci) k užití této mojí práce, a to včetně všech počítačových programů, jež jsou její součástí či přílohou a veškeré jejich dokumentace (dále souhrnně jen „Dílo“), a to všem osobám, které si přejí Dílo užít. Tyto osoby jsou oprávněny Dílo užít jakýmkoli způsobem, který nesnižuje hodnotu Díla a za jakýmkoli účelem (včetně užití k výdělečným účelům). Toto oprávnění je časově, teritoriálně i množstevně neomezené.

V Praze dne 30. března 2026

Abstrakt

Tato diplomová práce se věnuje návrhu a implementaci webové aplikace pro správu tanečních akcí a kurzů v oblasti společenských tanců. Vychází z analýzy současných způsobů vyhledávání a organizace akcí, které jsou často rozptýlené mezi více platformami a neposkytují dostatečný přehled o registracích ani účastnících. Na základě zjištěných nedostatků jsou formulovány požadavky a navržen systém a jeho uživatelské rozhraní podporující vyhledávání a filtrování akcí, správu událostí organizátory, registrace účastníků i práci s uživatelskými profily.

Implementace je realizována jako klient-server aplikace s frontendem v Reactu, backendem ve Spring Bootu (Kotlin), relační databází PostgreSQL a objektovým úložištěm MinIO pro média. Řešení je ověřeno kombinací manuálních testů, automatizovaných unit testů na frontendu i backendu a uživatelského testování s cílovou skupinou tanečníků a organizátorů akcí. V závěru práce jsou shrnuty dosažené výsledky v podobě funkčního prototypu a diskutovány možnosti dalšího rozvoje systému.

Klíčová slova společenské tance, webová aplikace, PostgreSQL, React, Spring Boot, Kotlin, full-stack

Abstract

This master's thesis focuses on the design and implementation of a web application for managing social dance events and courses. It is based on an analysis of current approaches to event discovery and organization, which are often fragmented across multiple platforms and do not provide sufficient visibility into registrations and participants. Based on the identified limitations, requirements are formulated, and both the system and its user interface are designed to support event search and filtering, event management by organizers, participant registrations, and user profile management.

The implementation follows a client-server architecture with a React frontend, a Spring Boot (Kotlin) backend, a PostgreSQL relational database, and MinIO object storage for media files. The solution is validated through a combination of manual testing, automated unit tests on both frontend and backend, and user testing with the target group of dancers and event organizers. The thesis concludes by summarizing the achieved results in the form of a functional prototype and discussing possible directions for further system development.

Keywords social dancing, web application, PostgreSQL, React, Spring Boot, Kotlin, full-stack

Obsah

1 Úvod **1**

2 Společenské tance **3**
2.1 Co jsou to společenské tance . . . 3
2.2 Druhy společenských tanců . . . 4
2.2.1 Salsa . . . 4
2.2.2 Bachata . . . 4
2.2.3 Zouk . . . 5
2.3 Cesta tanečníka ve světě společenských tanců . . . 6
2.4 Existující prostředky pro vyhledávání a organizaci událostí . . 6
2.4.1 Webové stránky tanečních škol . . . 6
2.4.2 Sociální sítě . . . 7
2.4.3 Mobilní aplikace . . . 9
2.4.4 Shrnutí - motivace pro vytvoření nové aplikace . . . 9

3 Analýza a návrh systému **10**
3.1 Požadavky . . . 10
3.1.1 Must have (musí mít) . . . 11
3.1.2 Should have (mělo by mít) . . . 16
3.1.3 Could have (mohlo by mít) . . . 18
3.1.4 Won't have (nebude mít) . . . 19
3.2 Návrh systému . . . 20
3.2.1 Návrh databáze . . . 20
3.2.2 Server - návrh backendu . . . 24
3.2.2.1 NestJS . . . 24
3.2.2.2 Spring Boot . . . 24
3.2.3 Klient - návrh frontendu . . . 25

4 Návrh uživatelského rozhraní **27**
4.1 Wireframing . . . 27
4.1.1 Figma . . . 28
4.2 Design systém . . . 28
4.2.1 Material UI . . . 29
4.2.2 Ant Design . . . 29
4.2.3 Chakra UI . . . 29
4.2.4 Shadcn UI . . . 30

4.3 Tmavý režim 30
4.4 Theme aplikace 30
4.5 Návrh jednotlivých částí aplikace 31
4.5.1 Navigační prvky 31
4.5.2 Domovská stránka 31
4.5.3 Nadcházející akce 32
4.5.4 Detail akce 34
4.5.5 Modal pro publikování akce 36
4.5.6 Profil uživatele 36
4.5.7 Moje akce 38
4.5.8 Vytvořit akci 40
4.5.9 O aplikaci 41
4.5.10 Stránka s registrovanými uživateli 41
4.5.11 Přihlašování a odhlašování z aplikace 41

5 Implementace 43
5.1 Server 43
5.1.1 Přihlášení uživatele 43
5.1.1.1 Inkrementální autorizace 44
5.1.1.2 Proces přihlášení 44
5.1.2 Registrace uživatele 46
5.1.3 Integrace s Google Forms 48
5.2 Frontend 51
5.2.1 API na lokace 51
5.2.1.1 Google places API (New) 52
5.2.1.2 Photon 52
5.2.2 Formuláře 53
5.2.3 Načítání dat z API 53
5.2.4 Routování 53
5.3 Databáze 54

6 Testování 55
6.1 Testování 55
6.1.1 Typy testů 56
6.1.2 Způsob testování 56
6.2 Implementace testů pro jednotlivé části systému 56
6.2.1 Frontend 57
6.2.2 Backend 57
6.3 Uživatelské testování 58
6.3.1 Scénáře pro běžného uživatele 58
6.3.2 Scénáře pro organizátora akcí 59
6.3.3 Uživatelé účastnící se testování 60
6.3.4 Výsledky testování 61

7 Závěr **62**

A Nějaká příloha **64**

Obsah příloh **70**

Seznam obrázků

3.1 ER diagram navržené databáze. 23

4.1 Finální design navigační lišty aplikace 32
4.2 Návrh domovské stránky 33
4.3 Návrh pro zobrazení nadcházejících akcí na desktopovém zařízení 34
4.4 Finální design detailu akce 37
4.5 Výsledný profil uživatele 38
4.6 Finální implementace stránky "Moje akce" 39

5.1 Stavový diagram pro registrace. 47

Seznam tabulek

Seznam výpisů kódu

5.1 Funkce pro generování url pro přihlášení přes Google OAuth . . 45
5.2 Příklad části struktury odpovědi pro endpoint vracející registrace 47
5.3 Příklad definice routy pro profil uživatele 53

6.1 Příklad unit testu pro frontend 57

Seznam zkratek

API	Application Programming Interface
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model–View–Controller
OAuth	Open Authorization
SQL	Structured Query Language
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience

Kapitola 1
Úvod

Společenské tance (často označované anglickým termínem *social dance*) jsou tance, které se tančí především za účelem zábavy a společenské interakce. V posledních letech nabývají na popularitě a s tím roste i množství tanečních kurzů, workshopů a akcí. Tanců je několik, avšak s ohledem na cílovou skupinu se tato práce zaměřuje na salsu, bachatu a zouk.

Rostoucí nabídka událostí je pro taneční komunitu přínosná, současně však zvyšuje nároky na orientaci v dostupných možnostech. Pro začínající tanečníky bývá obtížné zjistit, kde a kdy se konají vhodné akce a jaké jsou jejich podmínky (úroveň, typ události, cena či lokalita). Zkušenější tanečníci naopak často hledají konkrétnější akce podle stylu nebo místa a potřebují rychle porovnat více možností. Z pohledu organizátorů je klíčová především přehledná správa událostí a registrací účastníků, a to i v případě, že jsou registrace sbírány prostřednictvím Google formulářů.

V praxi jsou informace o událostech často rozptýlené napříč více kanály. Část organizátorů zveřejňuje akce na webových stránkách nebo v kalendářích tanečních škol, významná část komunikace probíhá na sociálních sítích (typicky Facebook a Instagram) a registrace bývají řešeny externě, například přes Google Forms či zprávy. Tato roztříštěnost vede k tomu, že tanečníci musí události složitě dohledávat, a i po registraci mohou postrádat jasné potvrzení nebo přehled o svých přihláškách. Organizátorům naopak chybí jednotné místo pro prezentaci akcí a nástroje pro přehlednou administraci účastníků.

Tuto problematiku se snaží řešit tato diplomová práce, jejímž tématem je návrh a implementace webové aplikace pro správu společenských tanečních akcí a kurzů, která by spojovala tanečníky a organizátory. Nápad na téma vznikl z osobního zájmu autorky o společenské tance a z pozorování, že existuje mezera na trhu pro platformu, která by poskytla centralizovaný a přehledný přístup k událostem i jejich správě.

Cílem této práce je navrhnout, implementovat a vhodně otestovat webovou aplikaci pro správu společenských tanečních akcí a kurzů, která tanečníkům usnadní vyhledávání událostí a organizátorům umožní přehlednou správu akcí

a registrací účastníků.

Kapitola 2
Společenské tance

V následující kapitole je uvedena problematika společenských tanců, je vysvětleno co to jsou společenské tance, jaké druhy společenských tanců existují a jaké jsou jejich charakteristiky. Následně je analyzováno, jaké nástroje a technologie uživatelé využívají pro vyhledávání kurzů a akcí spojených s těmito tanci a jaké nástroje používají organizátoři pro organizaci a propagaci těchto akcí. Nakonec je popsána motivace pro vytvoření aplikace, která by uživatelům vyhledávání usnadnila.

2.1 Co jsou to společenské tance

Společenské tance (více známé jako nepřekládané "social dance") jsou tance, které se tančí především kvůli zábavě a společenské interakci mezi lidmi, na rozdíl od jiných forem tanců, které se zaměřují na soutěžení. Avšak je důležité poznamenat, že některé taneční akce nebo festivaly začali zahrnovat i možnost zúčastnit se soutěží. Soutěže ovšem bývají jinak postavené než klasické taneční soutěže, protože partneři většinou bývají náhodně přiřazeni. Tudíž tanečníci nemají možnost si s partnerem předem nacvičit choreografii, ale musí se spolehnout na své schopnosti improvizace a komunikace během tance. [1]

Společenské tance bývají nejčastěji párové tance, kde partneři spolu komunikují prostřednictvím pohybů a vedení. Jeden z partnerů obvykle bývá označován jako "vedoucí"(leader) a druhý jako "následující"(follower). Leader vede tanec a určuje směr a styl pohybu, zatímco follower reaguje na jeho vedení a doplňuje tanec svými vlastními pohyby. Co je na tanci zajímavé, je to že tyto tance se tancují většinou improvizovaně, což znamená, že neexistují pevně stanovené kroky nebo choreografie, které by museli tanečníci dodržovat. Místo toho leader vymýšlí kroky a pohyby na místě a pomocí správného vedení umožňuje followerovi správně reagovat. [1]

2.2 Druhy společenských tanců

Existuje mnoho druhů společenských tanců, které se liší svým stylem, rytmem a kulturou, ze které pocházejí. Já jsem se rozhodla pro účely mé diplomové práce se zaměřit na druhy tanců se kterými se nejvíce setkávám v mé taneční komunitě. Jedná se o následující tance:

- Salsa
- Bachata
- Zouk

V následující sekci popíši charakteristiky těchto tanců. Samozřejmě můžeme zmínit i další tance, se kterými se člověk taky může setkat, jako například Kizomba, Swing, Reggaeton a další, ale pro účely této práce jsem se rozhodla zaměřit pouze na ty tři, které jsem zmínila výše, protože jsou mi nejvíce blízké, setkávám se s nimi nejčastěji a tanečníci se jterými konzultuji, tyto styly tančí. Představení těchto tanečních stylů pomůže lépe porozumět kontextu společenských tanců a ukáže, jak se mohou vzájemně prolínat na různých akcích.

2.2.1 Salsa

Je latinsko-americký tanec s kubánskými kořeny vyznačující se rychlým tempem, energickými pohyby a výraznou rytmikou. Tanec působí živě a radostně, což z něj činí vhodnou volbu pro společenské události a taneční večery. [2]

Tento tanec vznikl zhruba ve 30. letech 20. století na Kubě a později se rozšířil do dalších částí Latinské Ameriky. Salsa se následně kvůli migraci rozšířila i do Spojených států, kde byla braná jako populární zábava. Díky tomuto se salsa různě proměňovala a vyvíjela, což vedlo k vytvoření různých stylů salsy. [3]

Salsa se většinou tančí na hudbu ve čtyřčtvrtečním taktu na doby 1, 2, 3 a 5, 6, 7, přičemž doba 4 a 8 jsou pauzy (avšak některé styly salsy mohou mít odlišné rytmus jako například New York salsa, která se tančí na doby 2, 3, 4 a 6, 7, 8). [4]

Z mé zkušenosti je salsa často na akcích ruku v ruce s bachatou, například v jedné místnosti se tančí salsa a v druhé místnosti se tančí bachata. Také se dějí tzv. SBK party [5], což znamená, že se na party tančí salsa, bachata a kizomba.

2.2.2 Bachata

Bachata je tanec pocházející z Dominikánské republiky, který je rozšířený po celém světě. Je známá především jako sensuální tanec. V tomto tanci se tančí sekvence kroků na 8 dob, v originálu se tančí do čtverce, ale v západním světě se začalo do strany, což je jednodušší. V obou případech se tančí 3 kroky a

poté následuje tzv. ”tap”, což je krok, kdy se noha zvedne a poklepe o zem (často doprovázená bokem do strany), ale nedělá se krok. [6]

Bachata se dělí na několik stylů, přičemž tanečníci často kombinují různé styly dohromady. Chtěla bych zmínit ty nejběžnější styly, které se tančí v mé komunitě, jako například [7]:

- Dominikánská bachata - tento styl je ve své původní podobě jednou z nejjednodušších, ale zároveň nejživějších forem tohoto tance. Tento styl se vyznačuje hravým footworkem a pohybem boků do strany podle rytmu hudby. Tento styl neobsahuje komplikované figury, což ho činí vynikajícím pro začátečníky.

- Bachata Moderna - nazývána také jako ”moderní bachata”je styl, který mixuje tradiční bachatu s jinými tanečními styly, jako je například salsa. Tento styl se především vyznačuje většími pohyby, dynamičtější projev a rychlé otočky. Také obsahuje více záklonů než bychom třeba shledali u původní bachaty.

- Sensual - tento styl vznikl ve Španělsku díky tanečníkům Korke Escalona a Judith Cordero za cílem přidat do bachaty více výrazů a emocí. Tento styl kombinuje prvky moderní bachaty s kroky brazilského zouku. Typickými prvky toho stylu jsou vlnící se pohyby těla a izolace (oddělené pohyby různých částí těla). Tanečníci obvykle tančí blízko sebe a pohybují se jako jeden celek, což navozuje emotivní a romantickou atmosféru.

2.2.3 Zouk

Je partnerský tanec, pocházející z Brazílie. Tento tanec vznikl v počátcích 90. let 20. století a byl ovlivněn karabskou zouk hudbou a tancem lambada. Během let se zouk vyvinul do několika stylů, přičemž se dá tančit nejenom na zouk hudbu, ale i například na hip hop, R&B, pop hudbu a další. Zouk se obecně vyznačuje vedením pomocí celého těla, nakloněné (říká se off-axis) pozice, otočky a náklony partnerky. Charakteristické pro zouk jsou také pohyby hlavy partnerky (říká se jim tzv. “hair movement”), které vznikají nakláněním followera při vedení. Tyto pohyby jsou dle mého názoru jedním z nejzajímavějších aspektů tohoto tance a zároveň i jedním z nejtěžších, protože vyžadují velkou kontrolu nad tělem a správné vedení. Zároveň jsem si všimla, že se tyto figury začaly objevovat i v bachatě, přičemž podle mého názoru nejsou vůbec jednoduché na provedení. [8]

Taneční styly, které se vzájemně kombinují, se označují pojmem „fusion“ [9].

2.3 Cesta tanečníka ve světě společenských tanců

Lidé se do světa společenských tanců často dostávají z různých důvodů. Z mojí zkušenosti nejčastěji prostřednictvím přátel, kteří již tančí, sociálních sítí, kde jsou propagovány taneční workshopy a kurzy, nebo prostřednictvím videí na platformách jako je YouTube. V této sekci popíšu, jak lidé obvykle začínají tančit, jaké akce navštěvují a jak se jejich taneční cesta vyvíjí, aby bylo možné lépe porozumět potřebám tanečníků při hledání informací o kurzech a akcích.

Většinou si člověk nejprve vybere konkrétní tanec, který ho zaujal, a následně začne vyhledávat kurzy, které se na tento tanec zaměřují. Prvním krokem proto bývá nalezení vhodného kurzu. Po pár lekcích většinou tanečníci začínají navštěvovat taneční akce, kde mohou tančit s různými partnery a zlepšovat své dovednosti v reálném prostředí. Na kurz člověk nemusí přijít v páru, protože se na lekcích partneři většinou střídají. Občas jsem se i setkala s tím, že kurz bývá koncipován tak, že nejprve proběhne výuková hodina/workshop, a poté následuje party, při které lidé procvičují co se naučili a improvizují. Zkušenější tanečníci se mohou účastnit různých tanečních festivalů, v rámci nichž probíhají workshopy a taneční večery. Některé taneční školy nebo organizace také nabízejí sérii workshopů nebo jednotlivých lekcí v rámci kurzu, na které se mohou tanečníci přihlašovat samostatně. Díky tomu si mohou vybrat pouze ty lekce, které je zajímají nebo jim časově vyhovují, místo přihlášení na celý kurz. Velkým zážitkem dle mého názoru je i cestování na taneční akce do jiných měst nebo zemí, kde člověk získá možnost poznat nové lidi a kultury prostřednictvím tance. [1]

2.4 Existující prostředky pro vyhledávání a organizaci událostí

V předchozí sekci jsem popsala jak se tanečníci dostávají do světa společenských tanců a jaké události mohou navštěvovat. V této sekci analyzuji, jaké nástroje a technologie uživatelé využívají k vyhledávání kurzů a akcí a jejich organizaci. Při tvorbě této analýzy jsem vycházela z vlastních zkušeností s vyhledáváním kurzů a akcí, zejména v oblasti bachaty v Praze, jelikož dlouhodobě na tyto akce docházím a jsem schopna lépe zhodnotit dostupné nástroje. Tato analýza byla zároveň konzultována s dalšími tanečníky působícími v této komunitě, kteří poskytli podněty vedoucí k jejímu zlepšení.

2.4.1 Webové stránky tanečních škol

Webové stránky byly pro mě prvním místem, kde jsem hledala informace o kurzech a akcích, když jsem chtěla začít tancovat. Většina tanečních škol má své vlastní webové stránky, kde zveřejňují informace o nabízených kurzech, termínech, cenách a dalších detailech. Tyto stránky často obsahují i kalendář akcí,

kde jsou uvedeny nadcházející taneční večery a workshopy. Nicméně, nevýhodou je, že uživatel musí procházet webové stránky tanečních škol jednotlivě a stranou si poznamenávat informace o kurzech, což může být časově náročné a únavné. Navíc je možné, že nějaké organizace/spolky nemají vlastní webové stránky, ale působí jen na sociálních sítích, tudíž na ně uživatel nemůže narazit na webu.

Existují i stránky, které agregují informace z více tanečních škol a spolků. Některé tyto stránky jsou koncipované tak, že abyste mohli přidat událost, musíte uzavřít spolupráci s danou stránkou, což často zahrnuje i poplatek za zveřejnění anebo si berou provizi z prodeje vstupenek. Toto může být pro některé organizace odrazující faktor, což vede k tomu, že událostí na těchto stránkách není tak hojný počet. Jedna z příkladů těchto stránek je go & dance, která se hlavně zaměřuje na propagaci velkolepých tanečních festivalů a kongresů. Tyto akce bývají v řádech stovek eur, což je poměrně drahé v porovnáním kolik stojí běžné kurzy a taneční večery. Stránka má hezký design a je přehledná, avšak pokud se snažím najít akce konající se v Praze, tak mi stránka vrátí i akce konající se v jiných městech. Toto je pro mě poměrně frustrující, protože musím procházet všechny akce a hledat ty, které se konají v Praze.

Jako další příklad jsem nalezla stránku tanci.cz, která cílila na kurzy a události pořádané v ČR, avšak tato stránka se jeví jako nefunkční a nenalézá mi žádné kurzy.

2.4.2 Sociální sítě

Sociální sítě jsou dle mého pozorování nejběžnějším prostředkem k vyhledávání nadcházejících tanečních akcích a zároveň i slouží jako platforma pro komunikaci mezi tanečníky a organizátory. Dle mého pozorování je nejčastější platformou pro tento účel Facebook.

Na Facebooku organizátoři vytvářejí události, kde zveřejňují informace o nadcházejících akcích, jako jsou termíny, místa konání, ceny a další detaily. Většina z nich pro registrace používá buď napsáním zprávu organizátorovi anebo google formuláře. Registrace bývá občas trochu chaotická, protože vyplníte například google formulář, ale nemáte jistotu, že organizátor váš zájem o účast zaznamenal, protože vám nepřijde žádné potvrzení o registraci. Také se stává, že tím jak je organizování dáno přes zprávy, tak se populárním organizátorům špatně udržují čekací listiny. To může vést k demotivaci lidí, kteří se snaží do kurzu dostat.

Uživatelé mohou události na Facebooku sledovat, přihlásit se na ně a sdílet je se svými přáteli. Facebook také umožňuje organizátorům cíleně propagovat své události prostřednictvím placených reklam, což zvyšuje jejich viditelnost a dosah. Nicméně, nevýhodou je, že pokud sociální síť nevyhodnotí, že by daná akce byla pro uživatele relevantní, nezobrazí ji. V důsledku toho může uživatel přijít o informace o akcích, které by ho mohly zajímat.

Uživatelé musí procházet různé skupiny, stránky a události, aby našli re-

levantní informace o nadcházejících akcích, což může být časově náročné a frustrující. Například při hledání bachatových akcí na Facebooku v sekci událostí se mi stává, že pokud akce nemá v názvu slovo „bachata“, Facebook ji nezobrazí ikdyž se jedná o akci na které se bachata tančit bude. Rovněž se mi stává, že při zadání filtru podle polohy, například „Praha“, Facebook zobrazí i akce konané mimo zvolenou lokalitu. V důsledku toho může dojít k tomu, že začnu zkoumat akce, které se konají jinde, například v Brně, což představuje zbytečnou ztrátu času. Z mé zkušenosti velmi pomohlo, když jsem se na Facebooku propojila s tanečníky, protože mi to umožnilo vidět události, o které mají zájem a tímto se stalo pro mne jednodušší akce dohledávat.

Další sociální sítí, která se využívá pro tento účel, je Instagram. Na Instagramu organizátoři často sdílejí informace o nadcházejících akcích prostřednictvím příspěvků a příběhů. Uživatelé mohou sledovat účty organizátorů a získávat informace o nadcházejících akcích. Nicméně je zde ještě horší vyhledávání akcí a spíše se instagram využívá pro propagaci akcí a komunikaci s uživateli, než pro samotné vyhledávání.

Po té jsou tu i další platformy jako například Whatsapp nebo Messenger, kde se často vytvářejí skupiny pro tanečníky, kteří spolu komunikují a sdílejí informace o nadcházejících akcích. Tyto skupiny jsou často uzavřené a přístupné pouze pro členy, což může být nevýhodou pro nové tanečníky, kteří se snaží najít informace o akcích.

Obecně jsou sociální sítě velmi užitečné pro komunikaci a propagaci akcí, avšak nejsou ideální pro samotné vyhledávání, protože jejich primární funkcí je sdílení a komunikace, nikoli organizace či katalogizace tanečních akcí.

Dalším problémem je rozptýlenost informací. Některé akce jsou propagovány pouze na Facebooku, jiné pouze na Instagramu nebo prostřednictvím WhatsApp skupin. Tímto způsobem jsem musela procházet různé platformy, abych našla požadovanou informaci, přičemž někdy jsem si nepamatovala, kde jsem ji původně viděla. Navíc, pokud je akce zveřejněna na více platformách, mohou se informace stát nekonzistentními.

Například se mi jednou stalo, že jsem se přihlásila na akci na Facebooku a později jsem se přidala do WhatsApp skupiny související s touto akcí. Pro účast na akci však nebylo povinné se do skupiny připojit. Po té došlo ke změně času konání akce, ale informace byla zprostředkována pouze prostřednictvím WhatsApp skupiny, nikoliv na Facebooku. V důsledku toho, že většina lidí sledovala akci pouze na Facebooku, se mnozí účastníci dostavili pozdě a nebyli schopni se zúčastnit naplánovaného workshopu.

Mezi dalšími nedostatky patří že uživatel kromě svých přátel nevidí kdo se na akci přihlásil nebo jaký je poměr mezi leadery a followery (v případě, že se jedná o akci, kde je tato informace relevantní).

2.4.3 Mobilní aplikace

Mobilních aplikací, které by zahrnovali různé taneční akce, je poměrně málo. Při prohlížení jednotlivých aplikací jsem narážela na problém, že neobsahují skoro žádné akce a jeví se jako poměrně neaktivní. Poté mi jeden známý doporučil aplikaci "Only Dance", která se nejenom zaměřuje na taneční akce ale také funguje jako sociální síť pro tanečníky. Tato aplikace má poměrně hezký design a nabízí i funkcionality jako možnost vytvářet spojení mezi tanečníky nebo psát příspěvky, které celá komunita může vidět. Nicméně, nevýhodou je, že aplikace obsahuje pouze možnost přidat reakci na akci, že plánuju jít anebo že se mi akce líbí, ale neobsahuje možnost rozlišit role pro akce, kde by to bylo relevantní. Také nemohu vidět kdo na akci jde, pokud uživatele nemám ve spojení. Dále mi přijde škoda, že uživatel nemůže přidávat videa na profil, určitě by to usnadnilo hledání tanečního partnera. Co mě na aplikaci nepříjemně překvapilo je, že když se akce zruší, tak ji aplikace smaže, ale neoznačí ji jako zrušenou, což může být pro uživatele matoucí, protože s akcí již počítal a najednou se mu ztratí z přehledu, aniž by věděl, co se stalo.

2.4.4 Shrnutí - motivace pro vytvoření nové aplikace

V předchozích sekcích jsem popsala různé nástroje a technologie, které jsou využívány pro vyhledávání kurzů a akcí spojenými se společenskými tanci. Z mého pohledu je zřejmé, že existuje několik problémů, které uživatelé mohou zažít při hledání těchto informací. Tyto problémy zahrnují rozptýlenost informací, nekonzistentní informace o akcích, nedostatek přehledu o tom, kdo se na akci přihlásil a zda uživatelova akce vedla k úspěšné registraci. Motivací pro vytvoření nové aplikace je tedy snaha o zjednodušení procesu vyhledávání a registrace na taneční akce, a zároveň poskytnutí lepšího přehledu o akcích a účastnících.

Cílem je vytvořit platformu, která bude uživatelsky přívětivá, efektivní a bude sloužit jako centralizované místo pro všechny informace týkající se společenských tanců. Pokud by se podařilo aplikaci v budoucnu propagovat a získat dostatečnou uživatelskou základnu, tak by organizátoři mohli ušetřit za propagaci a uživatelé by měli přehled o všech akcích na jednom místě, což by mohlo vést k větší účasti na těchto akcích a tím i k rozvoji taneční komunity.

Kapitola 3

Analýza a návrh systému

Tato kapitola se zabývá specifikací požadavků na budoucí aplikaci, formulovaných na základě zjištění z předchozí kapitoly. Následně bude rozpracován návrh systému, na jehož základě budou zvoleny vhodné technologie a postupy.

3.1 Požadavky

Požadavky jsou důležitou částí vývoje softwaru, protože definují, co má systém dělat a jaké funkce by měl poskytovat uživatelům. Požadavky jsem definovala pomocí MOSCoW metody, která rozděluje požadavky do čtyř kategorií: Must have, Should have, Could have a Won't have. Metodu vytvořil D. Richardson při práci v Oracle za účelem pomoci prioritizovat požadavky pro vývoj softwaru.[10]

Tato metoda mi pomohla určit priority jednotlivých požadavků a zaměřit se na ty nejdůležitější funkce pro první verzi aplikace. V následujících podsekcích rozdělím požadavky podle této metodiky a ke každému požadavku přidám popis jeho významu a důležitosti pro budoucí aplikaci.

U každého požadavku jsem rozhodla, zda se jedná o funkční nebo nefunkční požadavek. Funkční požadavky – označené symbolem F a číslem před názvem požadavku (např. F1, F2) – popisují konkrétní funkce a chování systému (např. co se má stát po tom co uživatel klikne na tlačítko). Nefunkční požadavky – označené symbolem N a číslem před názvem požadavku (např. N1, N2) – specifikují vlastnosti systému (např. zda má být aplikace responzivní nebo jak rychle má zpracovávat transakce). [11]

Veškeré požadavky byly tvořeny na základě konzultací s taneční komunitou, analýzy stávajících řešení a mých vlastních zkušeností, jakožto člověka, který se aktivně účastní tanečních akcí a setkává se s problémy, které způsobují frustraci u tanečníků a organizátorů. Tyto požadavky jsou navrženy tak, aby řešily konkrétní problémy a zlepšovaly celkovou zkušenost s hledáním a organizací tanečních akcí.

3.1.1 Must have (musí mít)

Tyto požadavky jsou nezbytné pro základní funkčnost aplikace. Bez nich nedává nasazení aplikace smysl. Tudíž se jedná o požadavky, na které budu klást větší důraz a dávat jim nejvyšší prioritu při vývoji. [10]

F1 - Zobrazení nadcházejících akcí. Aplikace musí být schopna zobrazit seznam nadcházejících akcí spolu s jejich základními informacemi jako:

- Název akce
- Jméno organizátora
- Datum a čas
- Místo konání
- Cena pokud je uvedena
- Tagy přibližující informace o akci, jako je typ tance, úroveň obtížnosti, zda se jedná o kurz, workshop nebo party a další relevantní informace.
- Počet přihlášených účastníků a počet uživatelů kteří mají o akci zájem
- Možnost zobrazit detailní informace o akci

Pokud uživatel není organizátor, má navíc možnost se na akci přihlásit anebo ji označit že ho zajímá. Události se navíc vrací po stránkách, aby nedocházelo k zbytečnému načítání velkého množství dat najednou, což by mohlo zpomalit aplikaci.

Seznam nadcházejících akcí umožní uživateli získat přehled o tom, jaké akce se budou konat v blízké době a zároveň každá z nich obsahuje stručně sepsané informace. Na základě těchto informací se uživatel může rozhodnout, že chce vědět více o konkrétní akci a zobrazit její detail.

F2 - Filtrování nadcházejících akcí. Aplikace umožní uživatelům filtrovat nadcházející akce podle různých kritérií, jako jsou:

- Název akce
- Typ tance (swing, salsa, tango, atd.)
- Typ akce (kurz, workshop, party, atd.)
- Lokalita -filtrování na úrovni města nebo státu

Tento požadavek je nezbytný, protože umožní uživatelům snadno najít a prohlédnout si nadcházející taneční akce, aniž by museli, jak je to například na facebooku, hledat akce pomocí klíčových slov anebo doufat, že se jim akce zobrazí v jejich feedu na základě reakce jejich přátel. Vyfiltrované události se také budou vracet po stránkách jak je to popsáno v předchozím požadavku.

Tohle mě jako potencionálnímu uživateli například umožní odfiltrovat akce, které se konají pro mě v nedostupné lokalitě anebo akce které jsou zaměřené na jiný taneční styl než ten který preferuji.

F3 - Přihlášení uživatele pomocí Google účtu. Aplikace musí umožnit uživatelům přihlásit se pomocí svého Google účtu, což zjednoduší proces registrace a přihlášení, a zároveň umožní využít funkce spojené s Google službami, jako je například integrace s Google Forms. Zároveň nám bude poskytnuto jméno a email uživatele, což usnadní uživateli vyplňování informací o sobě na profilu, protože tyto informace bude mít předvyplněné.

F4 - Odhlášení uživatele pomocí Google účtu. Aplikace musí umožnit uživatelům odhlásit se z aplikace, aby měli například možnost na stejném zařízení používat více účtů.

F5 - Detail události. Přihlášený uživatel by měl mít možnost zobrazit detailní informace o události. Tyto informace by měly zahrnovat:

- Základní informace o akci (název, datum a čas, místo konání, cena, odkaz na facebookovou událost)
- Popisek akce
- Informace o organizátorovi (jméno, odkaz na profil)
- Informace o přihlášených účastnících (počet přihlášených, počet uživatelů kteří mají o akci zájem, popřípadě v jakých rolích jsou uživatelé zastoupeni (leader, follower))
- Tagy přibližující informace o akci, jako je typ tance, úroveň obtížnosti, zda se jedná o kurz, workshop nebo party a další relevantní informace.

Požadavek byl sepsán na základě konzultací s tanečníky, kde jsme diskutovali, jaké informace bychom uvítali u detailu akce, abychom se mohli lépe rozhodnout, zda se na akci přihlásit nebo ne. Opakovaně se mi dostávalo odpovědi, že by bylo užitečné mít informace o tom, kolik lidí se již registrovalo a kolik lidí má o akci zájem, protože to může být indikátor kvality akce (čím více lidí se přihlásí, tím více mám možnost například v případě párového tance ozkoušet různé partnery a akci si tím pádem více užít).

F6 - Přihlášený uživatel může přidávat vlastní akce. Přihlášený uživatel by měl mít možnost přidávat vlastní taneční akce do aplikace. Tento požadavek umožní komunitě sdílet informace o různých akcích. Přidání akce je proces, který si klade za cíl strukturovaně uživateli umožnit zadat v přehledném formuláři všechny potřebné informace o akci nezbytné pro její zveřejnění v aplikaci. Formulář organizátorovi poskytuje rozhraní jehož výsledkem bude událost s dostatečnou informační hodnotou pro potencionální účastníky.

Mezi tyto informace patří:

- Název akce
- Datum a čas
- Místo konání
- Cena pokud je uvedena a měna ve které je cena uvedena
- Popisek akce
- Informace o akci (typ tance, úroveň obtížnosti, druh události)
- Kapacita akce
- Odkaz na facebookovou událost (pokud existuje) - to umožní uživatelům zkontrolovat další informace o akci, které nejsou uvedeny v aplikaci, a zároveň umožní organizátorům snadno propojit svou akci s již existující facebookovou událostí.

Uvedené informace o akci byli sestaveny po diskuzích s různými organizátory (ať už se jedná o lektory anebo lidi kteří organizují taneční večírky). Zároveň jsem se inspirovala informacemi, které jsou běžně uváděny v popiscích akcí na facebooku nebo na webových stránkách organizátorů.

Formulář s informacemi bude rozdělen do několika kroků, aby se uživatelé necítili zahlceni množstvím informací, které musí zadat. Vzhledem k tomu, že organizátor nemusí mít při vytváření události k dispozici všechny potřebné informace, případně je ještě nemusí chtít zveřejnit nebo si přeje nejprve zkontrolovat výslednou podobu detailu akce, nebude odeslání formuláře znamenat okamžité zveřejnění akce v aplikaci. Místo toho se vytvoří koncept události, který bude organizátorovi přístupný pro další úpravy a doplnění informací. Organizátor bude mít možnost kdykoliv se vrátit k úpravě konceptu a doplnění informací o akci, než ji zveřejní.

F7 - Registrace na událost. Tento požadavek umožní uživatelům přihlásit se na události přímo prostřednictvím aplikace, což usnadní organizátorům správu registrací a účastníkům jednoduchý způsob, jak se zúčastnit akcí. Tento požadavek je klíčový pro hlavní účel aplikace, kterým je správa registrací na taneční akce.

F8 - Organizátor může publikovat akci. Po vytvoření konceptu akce bude organizátor mít možnost ji publikovat, což znamená, že se akce stane viditelnou pro všechny uživatele aplikace.

Aplikace bude umožňovat 3 následné možnosti při zveřejnění akce, jak se uživatelé mohou registrovat na akci:

- Otevřená registrace - každý se může přihlásit bez omezení do kapacity akce
- Párová registrace - každý se může přihlásit do kapacity akce, ale musí uvést zda je leader nebo follower
- Registrace pomocí Google formuláře - organizátor může propojit Google formulář pro registraci účastníků přímo s událostí v aplikaci, což by usnadnilo správu registrací.

Tyto možnosti zveřejnění jsou organizátory velmi vítané, protože umožňují například registraci na workshop ladies styling, kde bývá kapacita omezená a není nutné uvádět, zda je účastník leader nebo follower. Zároveň jsou vhodné i pro párové akce, u nichž je důležité mít přehled o počtu leaderů a followerů.

Protože je Google Formulář jedním z nejčastěji používaných nástrojů pro registraci účastníků, je pro organizátory velmi přínosná také možnost propojení s Google formulářem.

Organizátor bude mít možnost nastavit jakým způsobem chce registrovat účastníky a zároveň zvolit další nastavení přihlašování, jako je například možnost schvalovat registrace účastníků manuálně.

F9 - Organizátor může upravovat akci. Tento požadavek umožní organizátorům upravovat informace o své akci i po jejím zveřejnění, což je důležité pro případ, že se změní nějaké detaily akce nebo pokud organizátor zapomněl uvést nějakou důležitou informaci při vytváření akce.

Dle mých pozorování je běžné, že se mění čas anebo místo konání akce, případně se doplňují informace o programu, které předtím nebyly k dispozici.

F10 - Přihlášený uživatel může označit akce, které ho zajímají. Uživatelé mají rádi možnost označit si akce, které je zajímají. Já osobně jsem na toto chování zvyklá z Facebooku, kde si označuji události, které mě zajímají, abych je mohla později snadno najít a zároveň sledovat jejich aktualizace.

Tento požadavek umožní uživatelům snadno si označit akce, o které mají zájem, což jim pomůže sledovat události, které by chtěli navštívit, a zároveň poskytne organizátorům cenné informace o tom, jaký velký zájem je o jejich akci.

F11 - Zobrazení seznamu akcí, které uživatele zajímají. Tento požadavek umožní uživatelům zobrazit seznam akcí, které si označili jako zajímavé, což jim pomůže se k nim snadno vrátit a případně se na ně přihlásit.

F12 - Zobrazení seznamu akcí na které je uživatel přihlášen. Tento požadavek umožní uživatelům zobrazit seznam akcí, na které jsou přihlášeni. Díky tomu můžou snadno sledovat akce, na které se zaregistrovali, stav jejich registrace a zároveň jim poskytne snadný přístup k detailům těchto akcí.

Tento požadavek je tanečníky vítán, umožní jim rychle zjistit, v jakém stavu jsou jejich registrace na různé akce, což je důležité zejména v případě, pokud jsou na čekací listině anebo registrace musí být potvrzena organizátorem.

F13 - Zobrazení seznamu akcí které uživatel organizuje. Tento požadavek umožní uživatelům zobrazit seznam akcí, které organizují, ať už se jedná o publikované nebo nezveřejněné akce. Tato funkce je důležitá pro organizátory, protože jim umožní snadno sledovat a spravovat své akce, a zároveň jim poskytne přehled o tom, jaké akce mají v plánu.

F14 - Uživatelský profil. Jelikož chceme aby aplikace měla komunitní charakter, tak je důležité umožnit uživatelům vytvořit si svůj profil, kde budou mít možnost sdílet informace o sobě a svých tanečních preferencích. Uživatelé by měli mít možnost upravovat své osobní údaje, jako je jméno za kterým vystupují v aplikaci, dále další informace jako jsou taneční styly které preferují, popisek o sobě a informace o jejich taneční úrovni.

F15 - Zobrazení seznamu registrovaných účastníků. Tento požadavek umožní organizátorům akcí snadno zobrazit seznam registrovaných účastníků, což usnadní plánování a organizaci událostí. K registraci budou ještě přidány další informace o účastnících vycházející z informací které poskytli při registraci doplněné o odkaz na jejich profil, pokud existuje.

F16 - Integrace s Google Forms. Google Forms jsou častým nástrojem pro registraci účastníků jak jsme se již zmínili v předchozí kapitole. Integrace aplikace s Google Forms by umožnila organizátorům akcí snadno spravovat registrace a získávat informace o účastnících přímo z aplikace. Organizátoři by mohli vytvořit Google formulář pro registraci účastníků a propojit ho s událostí v aplikaci, což by usnadnilo správu registrací a umožnilo organizátorům tak snadno sledovat počet přihlášených účastníků a další informace o nich.

F17 - Uživatel může zrušit svou registraci na událost. Tento požadavek umožní uživatelům zrušit svou registraci na událost, pokud se rozhodnou, že se jí nebudou moci zúčastnit. Tato funkce je důležitá pro flexibilitu uživatelů a umožní organizátorům lépe plánovat své akce.

F18 - Organizátor může zrušit akci. Tento požadavek poskytuje organizátorům možnost zrušit událost v případě neočekávaných komplikací nebo

situace, kdy se akce nakonec nebude konat.

Proces zrušení události se liší podle aktuálního stavu události. Pokud se akce nachází ve stavu konceptu, její zrušení povede k úplnému odstranění záznamu z databáze, jelikož dosud nebyla zveřejněna a není dostupná ostatním uživatelům.

V případě, že je událost již publikována, nedojde k jejímu fyzickému smazání, ale ke změně jejího stavu na „CANCELLED“. Taková akce zůstane nadále viditelná v aplikaci, bude však jednoznačně označena jako nekonající se. Zároveň se zruší všechny registrace účastníků na tuto událost.Tento přístup umožní transparetnost vůči uživatelům, kteří se již na akci přihlásili, a zároveň poskytne informaci o zrušení pro všechny ostatní uživatele, kteří například zvažovali účast na této akci. Pokud bychom publikovanou událost fyzicky smazali, mohlo by to vést k nejasnostem a zmatku mezi uživateli, kteří o té akci věděli a nechápali by, kam zmizela.

F19 - Organizátor může zamítat registrace. Tento požadavek umožní organizátorům zamítat registrace účastníků, pokud se rozhodnou, že daná osoba není vhodná pro účast na akci.

Požadavek byl definován s ohledem na potřebu dát organizátorům možnost odebrat již zaregistrovaného uživatele. Může například nastat situace, kdy se účastník choval nevhodně na předchozí akci, ale je již přihlášen na další. V takovém případě by měl mít organizátor možnost jeho registraci zrušit, aby se předešlo opakování nevhodného chování a zároveň bylo zajištěno bezpečné prostředí pro ostatní účastníky.

N1 - Aplikace je responzivní pro mobilní zařízení. Jelikož se předpokládá, že aplikace bude používána i na mobilních zařízeních, je důležité, aby byla responzivní a přizpůsobovala se různým velikostem obrazovky telefonu. Tablety neuvádím, kvůli tomu, že z mých pozorování se zdá, že většina uživatelů používá k prohlížení informací o akcích a k registraci na akce spíše mobilní telefony než tablety. Nicméně, pokud by se ukázalo, že je potřeba optimalizovat aplikaci i pro tablety, tak by bylo možné tento požadavek rozšířit i na tablety.

3.1.2 Should have (mělo by mít)

Tyto požadavky jsou důležité, ale nejsou nezbytné pro základní funkčnost aplikace. Jejich implementace může výrazně zlepšit systém avšak bez této funkce lze aplikaci nasadit. Mezi příklady takových požadavků patří zrychlení aplikace, oprava menších chyb anebo nové featury vylepšující systém. Tyto požadavky se zpravidla implementují po dokončení požadavků z kategorie ”Must have”.[10]

F20 - Úvodní stránka aplikace. Aplikace by měla mít úvodní stránku, která poskytne uživatelům přehled o tom, co aplikace nabízí, důvody pro její

použití a jak začít. Tato stránka by měla být atraktivní a informativní, aby motivovala uživatele k registraci a používání aplikace.

F21 - Možnost potvrzování registrace organizátorem. Organizátor bude mít možnost při zveřejnění akce zvolit, zda chce mít možnost potvrzovat registrace účastníků. Pokud organizátor tuto možnost aktivuje, bude povinen každou registraci schvalovat manuálně. Tento mechanismus mu poskytuje větší kontrolu nad seznamem účastníků a umožňuje individuálně rozhodovat o přijetí či odmítnutí přihlášky.

Schválení může být podmíněno například přijetím registračního poplatku nebo úhradou plné ceny akce. Dalším důvodem může být charakter události, například pokud je určena pouze pro tanečníky určité úrovně. Organizátor tak může zajistit, že se akce zúčastní pouze osoby odpovídající požadované úrovni či dalším stanoveným podmínkám.

Tento požadavek je v kategorii "Should have", protože i bez něj může aplikace fungovat, jelikož většina akcí umožňuje otevřenou registraci bez nutnosti schvalování. Nicméně, pro některé organizátory může být tato funkce velmi užitečná a zlepšit jejich zkušenost s aplikací, což je důvod, proč ji zahrnuji do této kategorie.

F22 - Uživatelé registrující nad kapacitní akci jsou umístěni na čekací listinu. Tento požadavek umožní uživatelům, kteří se pokusí zaregistrovat na akci, která již dosáhla své kapacity, být zařazeni na čekací listinu. Pokud se některý z přihlášených účastníků odhlásí, první osoba na čekací listině bude automaticky přesunuta do seznamu registrovaných účastníků a uživatelé zároveň uvidí v systému aktualizaci stavu své registrace.

Tato funkcionalita je velmi důležitá, protože umožňuje lépe využít kapacitu akce a naplnit její potenciál. Často se totiž stává, že se někteří registrovaní účastníci nakonec akce nezúčastní, aniž by svou neúčast organizátorovi oznámili – například proto, že se jim nechce psát zprávu. U párových akcí pak může docházet k nevyváženému počtu leaderů a followerů, což může být pro organizátora problematické. Díky čekací listině je možné tyto situace lépe vyrovnávat a zároveň dát šanci účastnit se akce i uživatelům, kteří se chtěli přihlásit až poté, co byla kapacita naplněna.

F23 - Zobrazení seznamu účastníků pro přihlášené uživatele. Tento požadavek umožní přihlášeným uživatelům zobrazit seznam účastníků, kteří se zaregistrovali na akci, na kterou jsou sami přihlášeni. Tato funkce může zvýšit zájem o akci. Například v situaci, kdy uživatel uvidí, že se na akci přihlásili jeho přátelé nebo známí, což ho může motivovat k účasti.

Je však potřeba počítat s tím, že ne každý uživatel bude chtít, aby jeho jméno bylo v seznamu účastníků veřejně zobrazeno. Proto by měla být tato funkce volitelná a uživatelé by měli mít možnost skrýt své jméno v seznamu účastníků, pokud si to přejí.

F24 - Přidávání médií k akci a profilu. Pro propagaci akcí a zlepšení uživatelského zážitku by bylo užitečné umožnit organizátorům přidávat k detailu akce média, jako jsou fotografie nebo videa. Tato média mohou potenciálním účastníkům pomoci lépe si představit, co mohou od akce očekávat, a tím zvýšit jejich zájem o účast.

Zároveň by bylo vhodné umožnit uživatelům přidávat média také do jejich profilu. Díky tomu by ostatní uživatelé mohli lépe poznat jejich taneční styl a úroveň. To může být obzvláště užitečné v případě akcí, které vyžadují určitou úroveň v daném tanečním stylu. Organizátor by tak mohl nahlédnout do médií přidaných k profilu uživatele a získat lepší podklad pro posouzení, zda jeho taneční dovednosti odpovídají požadavkům dané události.

F25 - Vytvoření opakované akce. Ve světě tanečních akcí se často vyskytují opakující se události, například pravidelné workshopy nebo taneční večírky. Tento požadavek by umožnil organizátorům snadno vytvářet opakované akce, což by ušetřilo čas a usnadnilo správu těchto událostí. Organizátor by mohl nastavit frekvenci opakování (denně, týdně, měsíčně) a další parametry, jako je počet opakování nebo datum ukončení opakování.

F26 - Možnost spravovat opakované akce samostatně. Jelikož se opakující akce mohou lišit obsahem (například jeden týden může být zaměřen workshop na ladies styling a další týden na musicality), tak by bylo užitečné, aby organizátoři měli možnost spravovat jednotlivé opakované akce samostatně. To by jim umožnilo upravovat informace o konkrétní akci, například změnit popisek nebo přidat nové tagy, aniž by to ovlivnilo ostatní opakované akce. Také by bylo vhodné, aby šly jednotlivé akce publikovat postupně či zrušit jednotlivé akce bez nutnosti zrušení celé série opakovaných akcí.

F27 - Stránka o projektu. Aplikace bude obsahovat stránku s informacemi o projektu, která bude obsahovat informace o tom, kdo stojí za projektem a za jakým účelem byl vytvořen. Tato stránka bude sloužit k budování důvěry mezi uživatelem a produktem.

N2 - Aplikace je uživatelsky přívětivá. Aplikace bude navržena tak, aby byla uživatelsky přívětivá a intuitivní. Uživatel by měl být schopen rychle porozumět tomu, jak se v aplikaci pohybovat a jak provést základní akce bez nutnosti studovat návod.

3.1.3 Could have (mohlo by mít)

Tento druh požadavků bývá označován jako ”nice to have”a zahrnuje funkce, které by mohly být přidány v budoucnu, ale nejsou prioritou pro první verzi aplikace. Oproti požadavkům z kategorie ”Should have”se liší tím, že jejich implementace nemá na aplikaci takový vliv. Těmto požadavkům bývá přiřazována

menší priorita a prostor dostávají hlavně požadavky z předchozích kategorií, které dokážou narůst i přes očekávaný scope. [10]

F28 - Notifikace. Aplikace by mohla posílat notifikace uživatelům, například o nadcházejících akcích, změnách v jejich registracích nebo změnách provedených v akci, na které jsou přihlášeni. Tato funkce by zlepšila uživatelskou zkušenost a pomohla uživatelům být informováni o důležitých událostech.

3.1.4 Won't have (nebude mít)

Tato skupina požadavků nebývá implementována v následující verzi aplikace, ale může být zvážena pro budoucí vývoj. Někteří mohou tyto požadavky za ty, které se nikdy nezrealizují. Proto jsou občas rozdělovány do dvou skupin, podle toho zda v budoucnu připadá v úvahu jejich implementace či nikoliv.[10]

F29 - Integrace s kalendářem. Tato funkce by umožnila uživatelům synchronizovat své registrace na akce s jejich osobním kalendářem, což by usnadnilo plánování a připomínání nadcházejících událostí.

F30 - Scraping událostí z webu. Tento požadavek jsem zamýšlela implementovat za cílem naplnit aplikaci obsahem obvzlášť zpočátku, kdy ještě nebude mít mnoho uživatelů, kteří by přidávali akce. Scraping událostí z webu by umožnil automatické získávání informací o nadcházejících akcích z různých webových stránek a jejich přidávání do aplikace. Nicméně, vzhledem k tomu, že scraping může být právně problematický a může porušovat podmínky používání některých webových stránek, jsem se rozhodla tento požadavek zatím nezahrnovat do vývoje aplikace. Také je problém ten, že tím že se snažíme získat informace z různých webových stránek, můžeme narazit na různé formáty a struktury dat, což by mohlo zkomplikovat implementaci této funkce a způsobit problémy s kvalitou dat v aplikaci.

F31 - Internalizace. Tato funkce by umožnila uživatelům přepínat jazyk aplikace, což by zlepšilo přístupnost pro uživatele z různých zemí. Jelikož se v této doméně používá hlavně angličtina, tak tento požadavek není zas tak důležitý a tudíž se nebude zatím implementovat.

F32 - Hodnocení a recenze akcí. Uživatelé by měli mít možnost hodnotit a psát recenze na akce, což pomůže ostatním uživatelům při rozhodování, zda se akce zúčastnit.

N3 - Offline režim. Tento požadavek by umožnil uživatelům přístup k informacím o akcích i bez připojení k internetu, což by bylo užitečné při omezeném přístupu k síti.

3.2 Návrh systému

V této sekci se zaměřím na návrh systému, který bude implementovat požadavky definované v předchozí části. Jelikož se jedná o rozsáhlou webovou aplikaci, rozhodla jsem se mít odděleného klienta a server, což nám umožní větší flexibilitu při vývoji a nasazení aplikace. Tento přístup nám také umožní snadněji škálovat jednotlivé části systému podle potřeby.

Konkrétně jsem zvolila architekturu MVC (Model-View-Controller), která nám umožní oddělit logiku aplikace od její prezentace a usnadní nám tak vývoj a údržbu systému. Model bude zodpovědný za správu dat a obchodní logiku, View bude zodpovědný za prezentaci dat uživatelům, a Controller bude zprostředkovávat komunikaci mezi Modelem a View. [12]

3.2.1 Návrh databáze

Při výběru databáze jsem uvažovala zda použít relační anebo NoSQL databázi. Jelikož se jedná o systém, který bude mít poměrně složitou strukturu dat s mnoha vztahy mezi různými entitami (například uživatelé, akce, registrace, atd.), rozhodla jsem se pro relační databázi. Relační databáze nám umožní snadno modelovat tyto vztahy a poskytne nám robustní nástroje pro správu dat a zajištění jejich integrity.

Konkrétně jsem se rozhodla použít PostgreSQL [13], což je open-source relační databáze, se kterou jsem již měla zkušenosti jak v praxi tak při studiu, takže se mi s ní bude dobře pracovat a zároveň nám poskytne všechny potřebné funkce pro správu dat v naší aplikaci.

Na obrázku 3.1 je zobrazen návrh databázového modelu aplikace. Model byl navržen s ohledem na požadavky definované v předchozí části. Databázový návrh systému byl realizován podle principů třetí normální formy (3NF), aby se minimalizovala redundance dat a zajistila konzistence informací. Pro jednoznačnou identifikaci entit jsou použity téměř ve všech případech UUID jako primární klíče, což usnadňuje propojení dat napříč různými tabulkami a umožňuje bezpečné slučování nebo replikaci dat. Opakující se události jsou řešeny prostřednictvím nadřazené tabulky **event_parents**, která odděluje logickou sérii událostí od jednotlivých termínů, což umožňuje flexibilní správu jednotlivých výskytů bez narušení celé série. Navrhnutý model obsahuje ve výsledku 19 tabulek, které reprezentují různé entity a jejich vztahy v systému.

users - Tabulka obsahuje základní autentizační informace o uživatelích systému. Ukládá e-mailovou adresu, identifikaci poskytovatele přihlášení (např. Google) a související autentizační tokeny. Každý uživatel je identifikován unikátním UUID.

user_profiles - Tabulka rozšiřuje základní údaje o uživateli o profilové informace, jako jsou jméno, příjmení, biografie, taneční úroveň, preferované ta-

neční styly a další relevantní informace. Každý záznam v této tabulce je propojen s odpovídajícím uživatelem v tabulce users pomocí cizího klíče (1:1 vztah).

user_media - Tabulka slouží k propojení uživatelských profilů s fotkami anebo videi. Jedná se o tabulku vytvořenou dekompozicí, která umožňuje přiřazení více mediálních souborů k jednomu uživatelskému profilu a zároveň umožňuje, aby jeden mediální soubor mohl být přiřazen více uživatelským profilům (M:N vztah).

user_dance_styles - Tabulka mapuje M:N vztah mezi uživateli a tanečními styly, které preferují. Uživatelé mohou mít více preferovaných tanečních stylů a každý taneční styl může být preferován více uživateli.

events - Tabulka představuje hlavní entitu systému a obsahuje informace o jednotlivých tanečních událostech. Ukládá název, popis, organizátora, lokaci, datum a čas konání, cenu vstupného a další atributy související s událostí.

event_parents - Tabulka slouží k reprezentaci nadřazených událostí u opakujících se akcí. Jednotlivé konkrétní termíny těchto událostí jsou následně evidovány v tabulce events, která obsahuje cizí klíč odkazující na tabulku event_parents.

event_registration_settings - Tabulka obsahuje nastavení registrace pro konkrétní událost. Umožňuje definovat způsob registrace, na základě níchž se odvíjí chování registrace pro danou událost.

registrations - Tabulka eviduje registrace účastníků na jednotlivé události. Obsahuje informace o uživateli, stavu registrace a případně jeho roli během události nebo odpovědi z Google formuláře, pokud je tento způsob registrace zvolen. Každá registrace je propojena s konkrétní událostí, avšak není povinné, aby byla propojena s uživatelem, protože se může jednat o registraci pomocí Google formuláře, kdy nemusí být uživatel v systému registrován.

events_media - Tabulka umožňuje přiřazení mediálních souborů k událostem, například fotografií nebo propagačních materiálů. Tabulka obsahuje také pořadí zobrazení médií.

locations - Tabulka uchovává informace o místech konání událostí, například název místa, adresu, město a stát. Jedna lokace může být sdílena více událostmi.

dancer_roles - Tabulka obsahuje seznam možných tanečních rolí, například leader, follower nebo role_rotation. Slouží jako referenční tabulka pro uživatelské profily i registrace na události.

dance_styles - Tabulka uchovává seznam tanečních stylů podporovaných systémem, například salsa, bachata nebo kizomba. Tyto styly mohou být přiřazeny uživatelům i událostem.

skill_levels - Tabulka definuje úrovně tanečních dovedností. Obsahuje název úrovně a pořadí, které umožňuje jejich hierarchické řazení.

event_types - Tabulka obsahuje typy událostí, například workshop, festival, soutěž nebo společenskou taneční akci.

currencies - Tabulka ukládá seznam měn používaných při definování ceny událostí, včetně jejich kódu, názvu a symbolu.

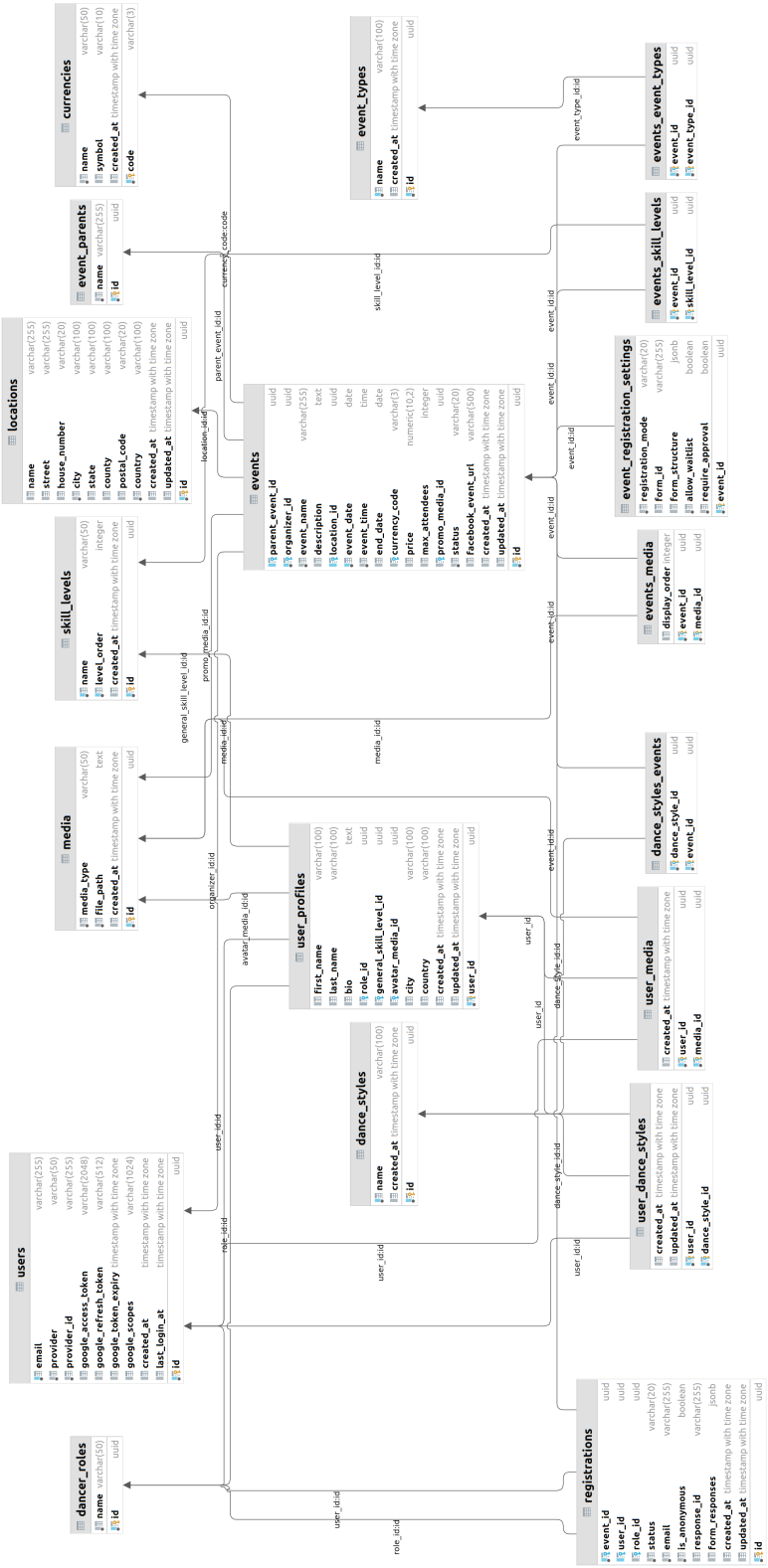
media - Tabulka uchovává informace o mediálních souborech, například obrázcích nebo videích, které mohou být použity v rámci systému. Každý záznam v této tabulce obsahuje informace o typu média, jeho umístění a kdy byl vytvořen.

dance_styles_events - Tabulka reprezentuje vztah M:N mezi tanečními styly a událostmi.

events_skill_levels - Tabulka propojuje události s úrovněmi tanečních dovedností, pro které je daná událost určena.

events_event_types - Tabulka reprezentuje vztah M:N mezi událostmi a jejich typy.

Dle požadavku .. bylo třeba ještě vymyslet jak ukládat obrázky a videa. Jelikož jsem je nechtěla ukládat přímo do databáze, ale využít nějaké úložiště, které je pro tento účel vhodnější. Jelikož videa můžou být v budoucnu většího rozsahu, rozhodla jsem se použít pro ukládání médií objektovou databázi [14], protože je pro tento účel vhodnější než tradiční relační databáze. Jelikož si chci databázi hostovat sama kvůli finančním prostředkům, rozhodla jsem se pro MinIO [15], což je open-source objektové úložiště. MinIO nám umožní snadno ukládat a spravovat média, jako jsou fotografie a videa, a zároveň nám poskytne škálovatelné řešení pro růst našeho systému.



Obrázek 3.1 ER diagram navržené databáze.

3.2.2 Server - návrh backendu

Backend bude zodpovědný za zpracování požadavků od klienta, komunikaci s databází a implementaci obchodní logiky. V následujících podsekcích se zaměřím na výběr frameworku pro backend podporující architekturu MVC.

3.2.2.1 NestJS

Pro vývoj backendu jsem zvažovala NestJS. Jedná se o framework umožňující vývoj efektivních, škálovatelných server-side aplikací postavených na Node.js. NestJS využívá moderní JavaScript ale plně podporuje TypeScript. Tento framework obsahuje aspekty jak z OOP tak i z funkcionálního a reaktivního programování. Jeho motivací bylo poskytnout vývojářům framework s architekturou inspirovanou framewrokem Angular, která usnadní vývoj a údržbu aplikací. samotný Nest.js používá Express jako defaultní HTTP server, ale poskytuje nad ním vyšší úroveň abstrakce a struktury. [16]

Z mé zkušenosti se backend aplikace psané v Node.js postrádají tuto abstrakci architektury a bývají psané za využití frameworku Express, který je dle mého názoru příliš jednoduchý a nenabízí tolik připravených funcionalit (např. ORM [17]), což v praxi může vést k nepřehlednému a těžko udržovatelnému kódu, zejména v případě větších aplikací.

Při výběru jsem vzala v úvahu jeho výhody a nevýhody. Mezi výhody patří:

- Mám zkušenosti s Node.js a TypeScriptem, takže jazyk frameworku pro mě nebude úplně nový
- Obsahuje instalovatelné balíčky jako např. ORM [17] pro namapování databáze na objekty, které nám umožní psát aplikace v architektuře MVC

Mezi nevýhody patří:

- Větší křivka učení z hlediska frameworku (ne jazyku) pro mě, protože jsem s tímto frameworkem ještě nepracovala, takže se budu muset nejdříve naučit jak s ním pracovat, což může zpomalit vývoj.
- Architektura Angularu mi v minulosti nevyhovovala, protože mi přišla příliš robustní a složitá, takže se obávám, že podobná architektura v NestJS mi také nebude vyhovovat.
- Přijde mi, že má menší komunitu než jiné frameworky, což může znamenat méně dostupných zdrojů a podpory při vývoji.

3.2.2.2 Spring Boot

Spring Boot je open source, microservice oriented framework pro vývoj Java webových aplikací.[18]

Je postaven na Spring Frameworku a zjednodušuje proces vývoje a nasazení aplikací tím, že poskytuje předkonfigurované nastavení. Spring Boot umožňuje

vývojářům snadno vytvářet samostatné, produkčně připravené aplikace. Narozdíl od Spring Frameworku, kde konfigurace a nastavení nového projektu vyžadují nějaký čas a znalosti. [19]

Mezi výhody Spring Bootu patří:

- Mám zkušenosti s Java (trochu i Kotlinem) a Spring Frameworkem, takže jazyk a některé koncepty frameworku pro mě nebudou úplně nové, což usnadní vývoj
- Velká a aktivní komunita, což znamená, že je k dispozici mnoho zdrojů a podpory pro vývoj [20]
- Osvědčený framework, který se používá na vývoj mnoha enterprise aplikací [20]
- Je postaven na MVC architektuře [21]
- umožňuje snadnou integraci s různými databázemi (PostgreSQL, MySQL, Oracle, ElasticSearch atd.) [22]

Mezi nevýhody [22] patří:

- Jedná se o opinionated framework, což znamená, že do jisté míry diktuje v mnoha spektech od konfigurace k vybrání knihoven jak má být psána
- Pro aplikace, které vyžadují nízkou latenci, může být Spring Boot příliš robustní a nesplňovat požadavky na výkon a dobu odezvy.

Vzhledem k tomu, že Spring Boot převážil výhodami, zejména díky mým zkušenostem s Java a Spring Frameworkem, jsem se rozhodla pro tento framework pro vývoj backendu.

Při rozhodování o výběru programovacího jazyka pro projekt jsem zvažovala Kotlin i Javu. Ačkoliv jsem měla s Kotlinem méně zkušeností než s Javou, nakonec jsem se rozhodla pro Kotlin. Tento moderní jazyk totiž přináší několik výhod oproti Javě, například výrazné snížení množství opakujícího se kódu díky data classes nebo extension functions a bezpečnější práci s hodnotami null. [23]

3.2.3 Klient - návrh frontendu

Jako architekturu pro frontend jsem zvolila Single Page Application (SPA), což znamená, že aplikace bude načítat všechny potřebné zdroje (HTML, CSS, JavaScript) při prvním načtení a následně bude dynamicky aktualizovat obsah bez nutnosti načítat nové stránky z serveru. [24]

Tento přístup umožní rychlejší a plynulejší uživatelský zážitek, protože nebude docházet k přerušení způsobenému načítáním nových stránek. Navigace v rámci SPA má díky tomu spíše charakter plynulého přepínání obrazovek,

podobně jako u desktopové anebo mobilní aplikace. Tím, že SPA načte jádro aplikace při prvotním načtení, tak může provést rychlejší přechody mezi jednotlivými obrazovkami, protože nebude muset znovu načítat a renderovat celý obsah stránky. [25]

Jako framework pro vývoj SPA jsem se rozhodla použít React. S tímto frameworkem mám již zkušenosti jak z profesního života, tak i z projektů během mého studia. To usnadní čas strávený na vývoji. Nehledě na fakt, že v porovnání s ostatními frameworky jakými jsou například Angular nebo Vue, je pro mě API Reactu přehlednější a jednodušší na používání. Tento framework je populární a široce používaný [26], což nahrává tomu, že existuje plno knihoven pro různé use cases, které nám mohou usnadnit vývoj a ušetřit čas.

Frontend bude zajišťovat komunikaci s backendem prostřednictvím REST API, které bude implementováno v backendu.

Kapitola 4

Návrh uživatelského rozhraní

Tato kapitola pojednává o návrhu uživatelského rozhraní pro aplikaci. Nejprve bude uvedeno, proč je pro nás uživatelské rozhraní důležité a jaké jsou naše cíle při jeho návrhu. Následně budou uvedeny pojmy se kterými kapitola pracuje. Nakonec na základě požadavků vydefinovaných v předchozí kapitole budou popsány návrhy uživatelského rozhraní, popřípadě jejich vizuální návrhy a finální UI implementace.

Uživatelské rozhraní je v našem případě velmi důležité, protože pokud chceme aby aplikace byla úspěšná, je potřeba aby jí do budoucna využívalo co nejvíce lidí. Proto je potřeba aby bylo uživatelské rozhraní jednoduché, přehledné a intuitivní. Zároveň ale chci dosáhnout toho, aby bylo líbivé a uživatelé bavilo se v něm pohybovat a používat ho.

4.1 Wireframing

Je to způsob, jak navrhovat uživatelské rozhraní na úrovni jeho struktury a rozložení. Wireframe je návrh layoutu stránky, který demonstruje které prvky stránka bude mít. Jeho cílem je přiblížit celému týmu a stakoholderům jak bude vypadat výsledný produkt ještě předtím, než se začne s jeho vývojem. Díky tomu se dá předejít zbytečným změnám v pozdějších fázích vývoje, které bývají nákladné. Wireframy se dají budˇ vytvářet ručně, nebo pomocí různých nástrojů jako je například Figma, Adobe XD, Sketch a další. [27]

Wireframy se dělí na tři kategorie:

- Low-fidelity wireframe - jsou jednoduché, často černobílé, bez detailů a zaměřují se na především na rozložení prvků na stránce a navigaci mezi nimi.
- Mid-fidelity wireframe - jsou podrobnější než low-fidelity, slouží k zpřesnění výsledného designu. Designéři do nich přidávají poznámky a konkrétnější obsah, testují různé varianty uživatelských toků a prvků rozhraní.

- High-fidelity wireframe - se už podobají raným verzím finálního produktu. Obsahují vizuální i interaktivní prvky, ale stále nemusí mít plnou funkčnost prototypu. [28]

V rámci projektu budu jednotlivé typy wireframů využívat podle aktuální potřeby. Jelikož však na projektu pracuji sama a nemám k dispozici UX designéra, zaměřím se především na low-fidelity a mid-fidelity wireframy, které mi umožní efektivně navrhnout strukturu a rozložení aplikace. Tvorba high-fidelity wireframů je podle mých zkušeností časově náročná, což při omezeném časovém rámci projektu není příliš vhodné. Některé návrhy budu tvořit přímo v kódu, protože si to mohu namockovat podle potřeby a rovnou ozkoušet jak bude návrh vypadat a fungovat, což mi přijde efektivnější než tvorba high-fidelity wireframů, které by mi zabraly zbytečně moc času.

4.1.1 Figma

Pro návrh uživatelského rozhraní jsem zvolila nástroj Figma. Figma je nástroj, který je v základní verzi zdarma a i v ní umožňuje nejenom jednoduše tvořit návrhy uživatelského rozhraní, ale také obsahuje řadu užitečných funkcionalit, které usnadnují jejich tvorbu (možnost přidávat komentáře, tvořit komponenty, prototypovat a mnoho dalšího). Já osobně mám ráda, že můžu vytvořit parent komponentu (například navigační menu) a následně její kopie vkládat na všechny stránky, které ji potřebují. Díky tomu, že jsou všechny tyto kopie propojené s parent komponentou, stačí upravit pouze parent komponentu a všechny její kopie se automaticky aktualizují. To je velmi užitečné, protože se dá ušetřit spousta času a zároveň se dá předejít chybám, které by mohly vzniknout při ruční úpravě každé kopie zvlášť. Dále se mi velmi dobře pracuje s tzv. auto layoutem, který umožňuje nastavit pravidla pro rozložení prvků na stránce, a díky tomu se dá snadno přizpůsobit návrh různým velikostem obrazovky. [29]

Navíc pro Figmu bývají již vytvořené tzv. UI kit, což jsou předpřipravené komponenty, které odpovídají určitým designovým systémům (například Material Design od Google). Díky tomu se design dá více přiblížit finálnímu vzhledu a zároveň se dá ušetřit spousta času, protože není potřeba tvořit komponenty od začátku, ale skládat je z již předpřipravených. [30]

Tyto důvody potvrdily, že Figma je pro mě ideálním nástrojem pro návrh uživatelského rozhraní pro tento projekt. Díky zkušenostem s tímto nástrojem jsem schopná efektivně navrhnout uživatelské rozhraní.

4.2 Design systém

Design systém je soubor pravidel a opakovaně použitelných prvků, které zajišťují jednotný vzhled i celkový dojem z produktu a uživatelského prostředí. Design systém tvoří například barvy zvolené pro produkty, ikonky co budou

použity, typografie, ale také způsob jakým budou fungovat interakce s uživatelem. Design systém nám pomáhá udržovat konzistentní vzhled a chování napříč různými částmi produktu, což zlepšuje uživatelský zážitek a usnadňuje orientaci v aplikaci. [31]

V následujících sekcích rozeberu jednotlivé knihovny s komponentami, které se dají využít jako design systém nebo minimálně základ pro vytvoření vlastního design systému. Jelikož jsem v předchozí kapitole zvolili jako technologii pro vývoj aplikace React, zaměřím se na knihovny, které jsou určeny pro tento framework.

4.2.1 Material UI

Material UI je knihovna implementující Material Design od Google pro React. Tato knihovna obsahuje velké množství předpřipravených komponent, které jsou navrženy tak, aby odpovídaly designovým principům Material Designu. Díky tomu se dá snadno vytvořit konzistentní a moderní uživatelské rozhraní. Knihovna je velmi rozšířená a má velkou komunitu, včetně přehledné dokumentace, což usnadňuje její používání. Mezi její nevýhody patří avšak to, že knihovna má velký bundle size, což může negativně ovlivnit výkon aplikace, a také to, že má svůj specifický vzhled, který nemusí být vhodný pro všechny typy aplikací. Já osobně mám s Material UI hodně zkušeností, protože jsem s ní pracovala na bakalářské práci a vím jak funguje. Podle mého pozorování nabízí nejvíce předpřipravených komponent, které jsou funkční a dobře se s nimi pracují. Avšak jsem tuto knihovnu zavrhla, protože se snažím vytvořit aplikaci s poutavějším designem a snažit se ohýbat Material UI není vhodné dle mého názoru, protože to nepůjde jednoduše a zároveň tím ztrácíme částečně smysl, proč chceme použít tento design system. [32]

4.2.2 Ant Design

Je to design systém a knihovna komponent pro React zaměřená především na enterprise aplikace. Komponenty v Ant Design jsou navrženy spíše pro interní firemní nástroje (například administrační systémy nebo dashboardy) než pro aplikace určené běžným uživatelům. Vzhledem k tomu, že vzhled komponent této knihovny se mi nelíbí a navíc se mi zdá, že je zaměřená spíše na jiné typy aplikací, než jakou chci vytvořit, jsem se rozhodla tuto knihovnu také nezvolit. [32]

4.2.3 Chakra UI

Je další kvalitní knihovna komponent pro React, která se zaměřuje na jednoduchost a přizpůsobitelnost. Chakra UI nabízí širokou škálu komponent včetně grafů. API této knihovny je intuitivní a umožňuje snadno přizpůsobit vzhled a chování komponent. Knihovna má také skvělou dokumentaci, ve které jsou

uvedeny příklady použití a možnosti přizpůsobení. Co mi však vadí na této knihovně je to, že její vzhled se mi tolik nelíbí a narazila jsem tam u některých komponent na jiné chování než bych chtěla, například u comboboxu se možnosti zakliknuté přidávají nad ním, což mi připadá nepřehledné. Díky tomuto faktu jsem se rozhodla tuto knihovnu nezvolit. [32]

4.2.4 Shadcn UI

Shadcn UI není knihovnou design systému, ale spíše sbírkou komponent pro React, které jsou navrženy tak, aby byly co nejvíce přizpůsobitelné a umožňovaly vytvářet vlastní design systém.[32]

Tato knihovna bývá používáná s Tailwind CSS, což je utility-first CSS framework, který umožňuje rychle a efektivně vytvářet vlastní designy. [33]

Shadcn UI samotná stojí na na Radix UI a Base UI knihovnách a dostylizovaná pomocí Tailwind CSS. Knihovna se do aplikace nestahuje jako celek, ale jednotlivé komponenty jsou přidávány přímo jako kód do projektu. To umožňuje v případě potřeby komponenty upravit přímo v rámci aplikace. Díky tomu lze design velmi flexibilně přizpůsobit konkrétním požadavkům projektu. Mě osobně se knihovna líbí i jak je nastylovaná v defaultním stavu, ale zároveň ocením možnost pohrát si s designem a přizpůsobit ho podle svých představ. Díky tomu jsem se rozhodla tuto knihovnu zvolit pro tvorbu uživatelského rozhraní v tomto projektu. [32]

4.3 Tmavý režim

Při implementaci jednotlivých komponent budu zároveň implementovat i verzi pro tmavý režim. Rozhodla jsem se implementovat tmavý režim na základě požadavku N2, který se týká uživatelské přívětivosti. Tmavý režim nenavrhuji jako samostatný design, ale spíše jako variantu světlého režimu. Ve figmě komponenty navrhuji jen ve světlém režimu, protože na návrh tmavého režimu bych musela vynaložit moc času.

4.4 Theme aplikace

Shadcn UI umožňuje nastavit globální theme pro celou aplikaci, což nám umožní definovat například barvy, které se budou používat v celé aplikaci. Díky tomu můžeme zajistit konzistentní vzhled napříč různými částmi aplikace a zároveň nám to usnadní přizpůsobení designu v budoucnu, protože pokud budeme chtít změnit například hlavní barvu aplikace, stačí upravit hodnoty v jednom místě. Pro tento projekt jsem se rozhodla nastavit theme Red, protože mi přijde, že se nejvíce hodí k tanečnímu tématu oproti jiným variantám. [34]

4.5 Návrh jednotlivých částí aplikace

V této sekci chci navrhnout jednotlivé části aplikace. U většiny částí bude přiložen návrh uživatelského rozhraní, ať už pomocí wireframu, nebo přímo implementace v projektu.

4.5.1 Navigační prvky

Nejprve je potřeba navrhnout navigační prvky, které se budou nacházet na každé stránce aplikace. Jedním z těchto prvků je navigační lišta umístěná v horní části obrazovky, která obsahuje název aplikace s logem a také následující odkazy nebo akce:

- Sekci Events, která po rozkliknutí/najetí na ni nabídne odkazy i s vysvětlením, co daná stránka obsahuje (All events - seznam nadcházejících akcí, My events - seznam akcí, na které se uživatel přihlásil, organizoval anebo označil že má zájem, Create event - formulář pro vytvoření nové akce)
- Sekci About, která po rozkliknutí uživatele přesměruje na stránku s informacemi o aplikaci
- Ikonu slunce/měsíce, která bude sloužit pro přepínání mezi světlým a tmavým režimem
- Ikonu profilu, pokud je uživatel přihlášený, která po rozkliknutí zobrazí informace kdo je přihlášený (jméno uživatele a email) a nabídne odkazy na profil uživatele a odhlášení z aplikace
- Odkaz pro přihlášení do aplikace v případě, že uživatel není přihlášený

Pokud jsme na stránce pro přihlášení do aplikace, navigační lišta bude obsahovat pouze název aplikace s logem. U mobilního zobrazení bude navigační lišta obsahovat pouze název aplikace s logem, přepínač na tmavý režim a ikonu pro otevření menu, které bude obsahovat odkazy/akce jako v desktopovém zobrazení. Navigační lištu jsem implementovala rovnou v kódu, protože poskládání a prototypování v nástroji Figma by mi zabralo zbytečně moc času. Finální design navigační lišty je znázorněn na obrázku 4.1.

Dále jsem navrhla patičku aplikace, která se bude nacházet v dolní části obrazovky a bude obsahovat pouze název aplikace s logem.

4.5.2 Domovská stránka

Domovská stránka bude propagovat celou aplikaci a její hlavní funkce. Bude obsahovat krátký popis toho, co aplikace umí a jaké jsou její hlavní funkce. U nich budou 2 tlačítka, jedno (výraznější) pro přihlášení do aplikace a druhé pro zobrazení nadcházejících akcí. Dále bude obsahovat sekci představující hlavní

Obrázek 4.1 Finální design navigační lišty aplikace

funkcionality aplikace a důvody, proč by ji uživatelé měli používat. Nakonec se zde budou nacházet 3 akce, které se konají v nejbližší době. Tento návrh jsem tvořila na základě požadavku F20.

Tuto stránku jsem navrhla v nástroji Figma jako low-fidelity wireframe jak je vidět na obrázku 4.2.

4.5.3 Nadcházející akce

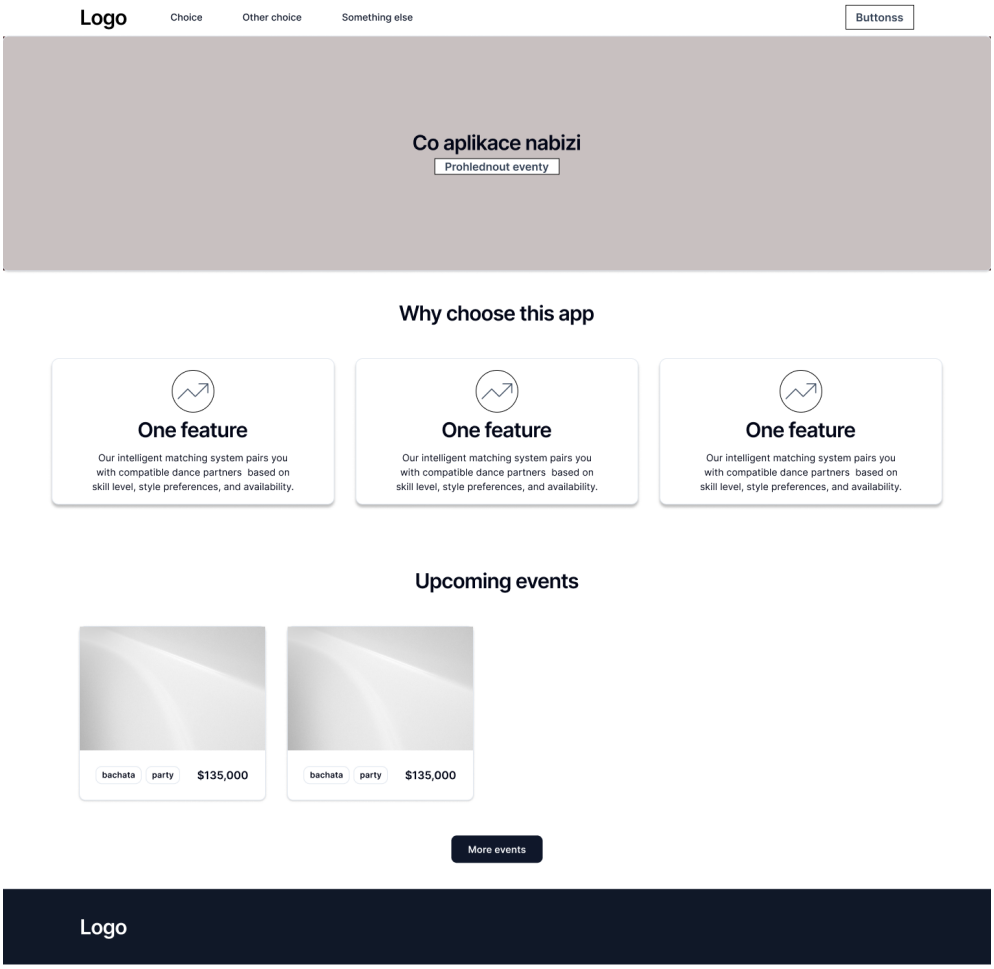
Tato stránka je velmi důležitá, protože hned po úvodní stránce, je stránkou, kterou uživatelé uvidí a budou se například rozhodovat zda se zaregistrují do aplikace.

Stránka se skládá ze sekce pro vyhledávání akcí a výčtu nadcházejících akcí. Pro vyhledávání jsem navrhla několik inputů, které umožní uživatelům filtrovat akce podle různých kritérií dle požadavku F2.

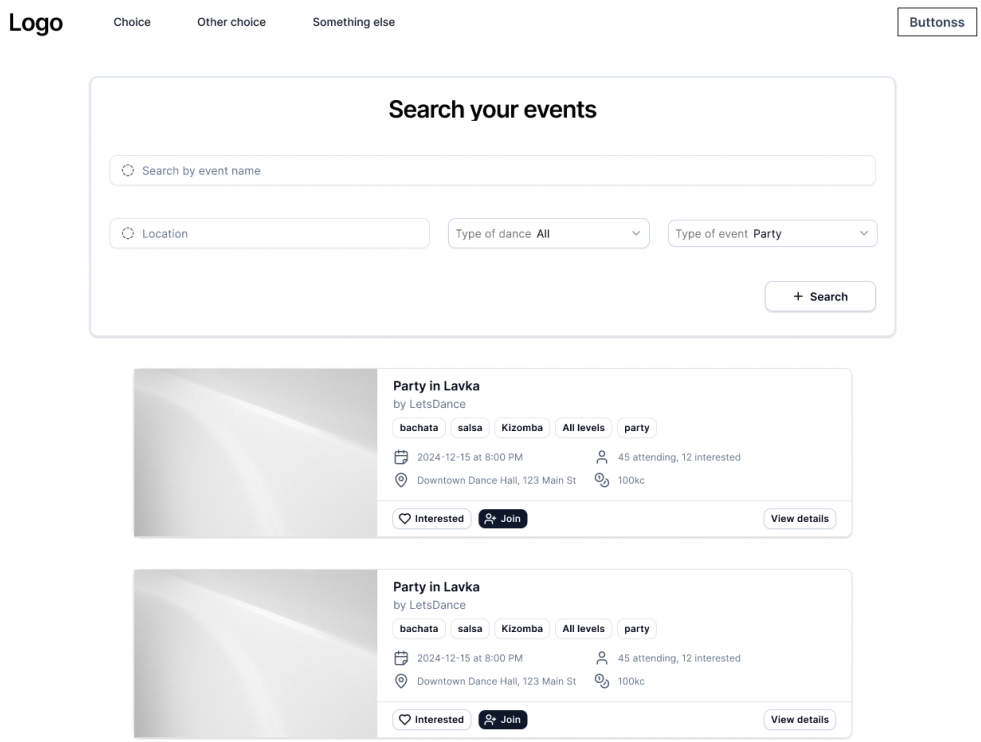
Pro filtrování jména akce jsem se rozhodla navrhnout dlouhý input, následně pro místo konání akce jsem navrhla combobox, který umožní uživatelům potom co začnou psát název místa, zobrazit možnosti, které odpovídají zadanému textu.

Pro typ akce a typ tance jsem navrhla také comboboxy avšak uživatel může vybrat více možností a pokud je tech možností více než tři tak se zobrazí pouze prvních pět a zbytek bude zobrazen jako "+X", kde X bude počet zbývajících možností. Po rozkliknutí komboboxu se uživateli zobrazí možnosti které zaškrtl. Tento výběr možností bude na mobilním zařízení implementován tak že vyjede nabídka (podobající se modalu) s možnostmi, které může uživatel zaškrtnout/odškrtnout.

Co se týče zobrazení nadcházejících akcí, navrhla jsem zobrazení ve formě karet, které budou obsahovat informace, jak jsou definované v požadavku F1. Kartu na desktopovém zařízení jsem pojala tak, že v levé části karty se bude nacházet obrázek akce anebo placeholder pokud obrázek není uvedený. Úplně nahoře v pravé části karty bude název akce, pod ním bude jméno organizátora a následně se budou nacházet zbylé informace o akci, obohacené o ikony. Úplně na spodu karty se budou nacházet akce, když je akce organizována uživatelem, bude se zde nacházet tlačítko přejít na detail akce, pokud uživatel organizátorem není, tak zde navíc budou akce jako přihlásit se na akci a označit že mám zájem o akci. Co mi chvíli dalo zabrat na návrhu bylo umístění tagů. Nakonec



Obrázek 4.2 Návrh domovské stránky



Obrázek 4.3 Návrh pro zobrazení nadcházejících akcí na desktopovém zařízení

jsem se rozhodla je umístit pod organizátora, protože jsem iterovala několik variant a tato se uživatelům zdála nejpřehlednější.

V mobilní variantě se obrázek a informace zobrazí nad sebou, aby se lépe přizpůsobily menší obrazovce.

Základ jsem si nakreslila do figmy (lze vidět na obrázku 4.3), ale finální design jsem implementovala rovnou v kódu, protože mi to přišlo efektivnější.

4.5.4 Detail akce

Tahle sekce byla pro mě nejtěžší na návrh, protože se zde nachází poměrně hodně informací a operací, které je potřeba nějak přehledně uspořádat. Při návrhu jsem vycházela z požadavku F5 a také z diskuzí s potenciálními uživateli, kteří mi poskytli cennou zpětnou vazbu ohledně toho, jak by měly být uspořádány. Při návrhu jsem se inspirovala webovými stránkami pro pořádání událostí, jako jsou například AllEvents, Eventbrite nebo Meetup, a také Facebookem, kde se tanečníci nejčastěji dozvídají o nadcházejících akcích. Na návrhu této stránky jsem chtěla věnovat více času, protože se jedná o jednu z nejklíčovějších stránek celé aplikace a právě prostřednictvím této stránky se uživatelé rozhodují, zda se na akci přihlásí, nebo ne.

Jako první co bude na stránce bude promo obrázek akce, který bude zob-

razený v celé šířce stránky. Pokud akce nemá promo obrázek, tak se zobrazí placeholder barva. Na tomto obrázku dole se bude nacházet název akce a pod ním klíčové informace o akci (tyto informace jsou sepsané v požadavku F5 jako základní informace o akci).

Potom stránka bude obsahovat tlačítka, která umožní pracovat s touto událostí. Tlačítka se liší podle toho zda je uživatel organizátorem akce, přihlášeným uživatelem, nebo uživatelem, který není přihlášený na akci. Pozici tlačítek jsem umístila pod promo obrázek s informacemi o akci, aby byli co nejvíce dostupné a nabádali uživatele k akci. Pokud je uživatel organizátorem akce, bude se zde nacházet tlačítko pro správu akce, které po rozkliknutí nabídne možnosti jako upravit akci a dále podle toho v jakém je akce stavu (smazat/zrušit, publikovat). Pokud uživatel organizátorem není, tak se zobrazují tlačítko pro přihlášení na akci a označení že mám zájem o akci.

Následně se budou nacházet na stránce tyto sekce (seřazeny od nejdůležitější po nejméně důležitou):

- Sekce s informacemi o účastnících akce jak je popsáno v požadavku F5.
- Popisek akce
- Detaily akce - zobrazení tagů, které byli popsané v požadavku F5
- Sekce s obrázky a videi týkající se akce
- Sekce s informacemi o organizátorovi akce

Zprvu jsem se snažila rozvrhnout jak a kde budou tyto zbývající sekce na stránce. Na mobilním zařízení mi přišlo logické dát jednotlivé sekce dát pod sebou, avšak na větších zařízeních docházelo k tomu, že se stránka stala příliš dlouhou a některé sekce nezabíraly celou šířku stránky, což mi přišlo nevyužité a nehezké z pohledu designu. Nakonec jsem se pro větší zařízení rozhodla pro rozvržení, které je zobrazeno na obrázku 4.4, kde se popisek akce a jeho galerie nachází v levé části stránky a zbytek sekcí se nachází v pravé části stránky. Toto rozvržení mi přijde přehlednější a zároveň využívá prostor na stránce lépe než kdybych všechny sekce dala pod sebe.

Co se týče návrhu jednotlivých sekcí, jejich vytvoření bylo poměrně jednoduché, s výjimkou sekce obsahující informace o účastnících akce. Původně jsem chtěla zobrazovat informace o tom, kolik lidí má o akci zájem a kolik lidí je již registrováno, přičemž jsem se inspirovala zobrazením těchto údajů na Facebooku. Zároveň jsem chtěla zobrazit také kapacitu akce, pokud je definována. Samotné zobrazení těchto údajů nebylo příliš složité, větší výzvou však bylo navrhnout způsob prezentace zastoupení jednotlivých rolí v případě akcí, které evidují rozdělení účastníků podle rolí.

Nakonec jsem zvolila design ve formě karty, která zobrazuje počet registrovaných účastníků a počet lidí, kteří o akci projevili zájem. Pokud má akce omezenou kapacitu, zobrazuje se zde také informace o počtu zbývajících míst,

která je vizuálně reprezentována pomocí progress baru (viz karta Who's Coming na obrázku 4.4).

V případě, že je u akce evidováno rozdělení účastníků podle rolí, karta navíc zobrazuje také jejich zastoupení. Jednotlivé role jsou uvedeny pod sebou v samostatných řádcích, kde je uveden název role a počet účastníků v dané roli. I zde je použit progress bar, který znázorňuje poměr zastoupení jednotlivých rolí, případně reflektuje celkovou kapacitu akce, pokud je stanovena.

Pokud akce nemá definovanou kapacitu a zároveň se při registraci nerozlišují role, zobrazuje se pouze počet registrovaných účastníků a počet lidí, kteří o akci projevili zájem.

Dále jsem chtěla uživatelům umožnit zobrazit konkrétní účastníky akce. Proto jsem navrhla, aby po kliknutí na tuto kartu došlo k otevření modálního okna se seznamem účastníků. Pokud je při registraci uváděna také role, modal rozdělí účastníky do samostatných sekcí podle jednotlivých rolí. V opačném případě se zobrazí pouze jeden seznam všech účastníků.

Sekce týkající se detailů akce a organizátora jsem navrhla také ve formě karet, aby byl celkový design stránky vizuálně konzistentní.

Sekce galerie obsahuje náhledy obrázků a videí, které jsou zobrazeny ve formě karet. Po kliknutí na konkrétní médium se obsah zobrazí v plné velikosti a v případě videa se zároveň spustí jeho přehrávání.

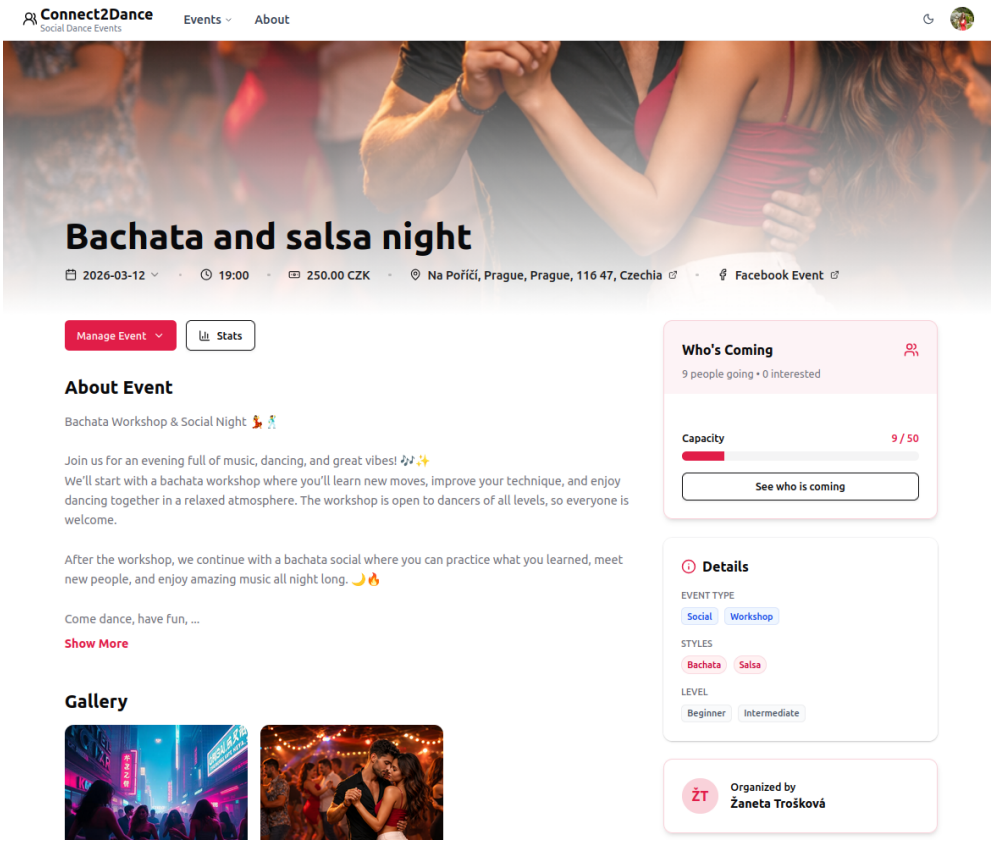
4.5.5 Modal pro publikování akce

Na rozdíl od jednoduchých operací, jako je smazání události, vyžaduje publikace akce speciální modal. Tento modal obsahuje formulář, ve kterém uživatel vybírá způsob zveřejnění akce, jak je specifikováno v požadavku F8. Modal zahrnuje název „Publish event“, krátký popis účinků zveřejnění a možnost výběru způsobu publikace. Pro všechny způsoby publikace lze nastavit, zda bude uživatel potvrzovat jednotlivé registrace účastníků, v souladu s požadavkem F21.

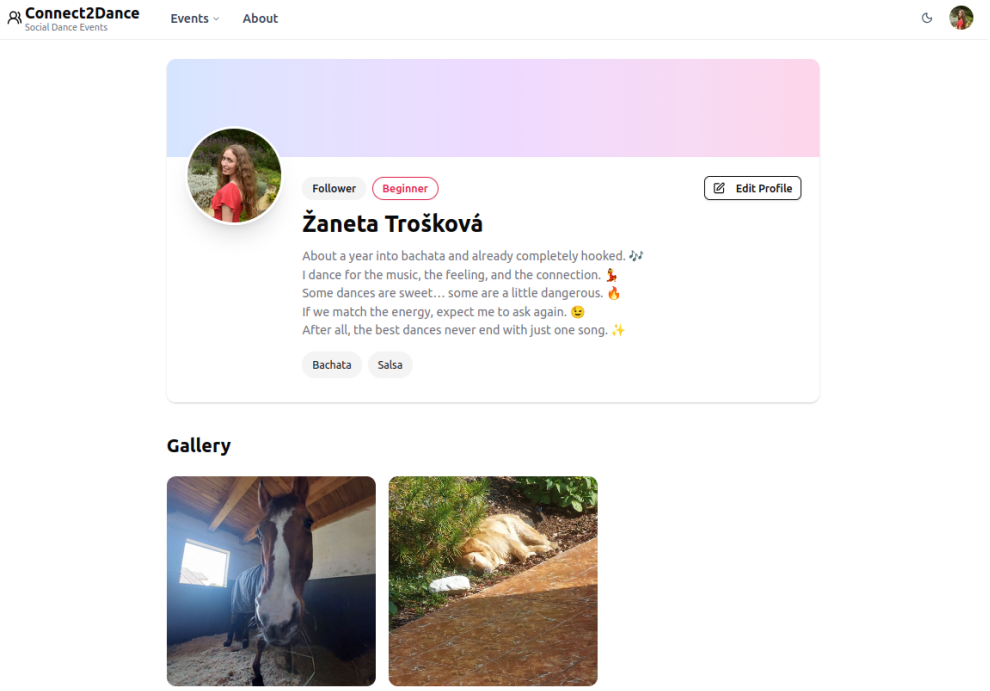
4.5.6 Profil uživatele

Profil uživatele jsem se snažila navrhnout tak, aby byl přehledný a obsahoval důležité informace o uživateli. Zároveň jsem chtěla, aby designem evokoval profil na sociální síti, protože jsou uživatelé na takové rozhraní zvyklí a bude pro ně přirozené se v něm pohybovat. Profil uživatele se skládá z dvou částí:

- Karta s informacemi o uživateli, která obsahuje informace jako jméno a příjmení uživatele, hlavní role v tanci(leader, follower, role rotation), avatar uživatele, obecná úroveň v tanci, popis o sobě a taneční styly, které uživatel tancuje. Pokud je profil přihlášeného uživatele, tak se zde navíc nachází tlačítko pro úpravu profilu.



Obrázek 4.4 Finální design detailu akce



Obrázek 4.5 Výsledný profil uživatele

- Galerie s obrázky a videi, které uživatel přidal do svého profilu. Galerie je řešena stejným designem jako u detailu akce.

Informace o tom co zobrazit v profilu uživatele jsem navrhla na základě požadavku F14 a galerii jsem navrhla na základě požadavku F24.

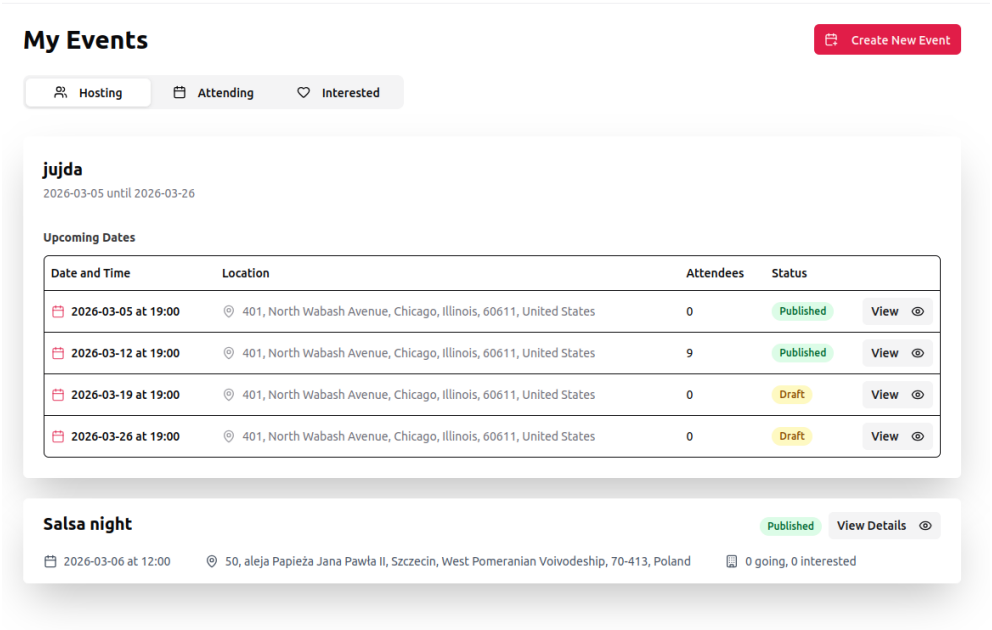
Pokud jde o upravování profilu, tak po kliknutí na tlačítko pro úpravu profilu se zobrazí formulář, který umožní uživateli upravit informace na profilu. Sekce galerie se v v tomto případě přesune do karty s formulářem, kde uživatel může přidat nové obrázky a videa, nebo odstranit stávající.

Design této komponenty jsem vytvářela přímo v kódu a jeho finální verze je znázorněna na obrázku 4.5.

4.5.7 Moje akce

Tato stránka bude obsahovat seznamy akcí, které uživatel organizuje, na které se přihlásil a které označil že ho zajímají na základě požadavků F11, F12 a F13. Jelikož se jedná o seznamy, které jsou vzájemně disjunktní, navrhla jsem zobrazení pomocí komponenty Tabs, která umožní uživateli přepínat mezi jednotlivými seznamy. Zároveň se bude nahoře objevovat tlačítko pro vytvoření nové akce a nadpis "My events".

Pokud se jedná o akci, která je opakovaná, tak se zobrazí název celé této



Obrázek 4.6 Finální implementace stránky "Moje akce"

série s rozsahem datumů od první do poslední akce, a pod tím se zobrazí pomocí tabulky jednotlivé výskyty této akce. Tabulka obsahuje datum a čas konání výskytu, místo konání, počet přihlášených uživatelů a status akce, popřípadě status uživatele, pokud jsme v sekci přihlášených akcí. Také se pro každý výskyt nachází tlačítko pro zobrazení detailu výskytu, které přesměruje uživatele na detail výskytu akce.

Pokud se jedná o akci, která není opakovaná, tak se zobrazí karta s názvem akce, datem a časem konání, místem konání, počtem přihlášených uživatelů a počtem uživatelů, kteří označili že je akce zajímá, a status akce, popřípadě status uživatele, pokud jsme v sekci přihlášených akcí. Také je zde tlačítko pro zobrazení detailu akce, které přesměruje uživatele na detail akce.

Přemýšlela jsem nad tím, že minimálně v případě jednorázové akce bych nemusela implementovat tlačítko pro zobrazení detailu akce, ale místo toho by se uživatel mohl dostat na detail akce kliknutím na kartu s informacemi o akci. Nakonec jsem se však rozhodla implementovat tlačítko pro zobrazení detailu akce, protože mi přijde přehlednější a zároveň je to konzistentní s tím, jak je to v případě opakovaných akcí.

Pokud uživatel nemá v dané sekci žádné akce, zobrazí se karta s informací, že sekce je prázdná, a s výzvou, aby uživatel začal procházet nadcházející akce a přihlásil se na některou z nich, nebo aby začal organizovat vlastní akci.

Na obrázku 4.6 je znázorněna finální implementace stránky v sekci událostí, které uživatel organizuje. Obrázek ilustruje zobrazení opakované akce s více výskyty a rovněž ukazuje, jak se zobrazuje jednorázová akce.

4.5.8 Vytvořit akci

Tuto část aplikace jsem navrhovala na základě požadavku F6, který specifikuje obsah formuláře pro vytvoření akce, a s ohledem na podporu opakovaných akcí dle požadavku F25.

Prvně jsem přemýšlela nad tím zda nemít jeden velký jednostránkový formulář. Toto řešení jsem navrhla protože z požadavku vyplívá, že pro vytvoření akce je potřeba zadat poměrně hodně informací, a já nechci aby se uživatelům zobrazoval příliš dlouhý formulář, který by je mohl odradit od vytvoření akce. Chvíli jsem ještě přemýšlela, že by sekce byly ve sbalitelných sekcích, což by mohlo napomoct k pocitu uživatele, že formulář není tak dlouhý. Nakonec jsem se rozhodla formulář rozdělit do několika kroků (každý krok představuje jednu stránku), protože se mi tento přístup jevil přehlednější. Informace tak lze rozdělit do logických celků, což uživatelům usnadňuje orientaci a zároveň snižuje pocit zahlcení množstvím údajů, které by jinak museli zadat najednou.

Existuje spousty způsobů, jak koncipovat aby každá sekce měla jednu stránku a mezi nimi se dalo přepínat. Pro inspiraci jsem procházela různé weby zaobírající se organizací akcí, jako například AllEvents. Na této stránce se používá pro přecházení mezi jednotlivými sekcemi formuláře boční nabídka. Toto řešení se mi moc nelíbí, protože mi přijde, že zbytečně zabírá místo na obrazovce a zároveň ve zbytku aplikace nepoužívám žádné boční menu, takže by to bylo nekonzistentní.

Nakonec jsem se rozhodla pro řešení, které bude obsahovat kartu s jednotlivými kroky formuláře, která bude umístěna uprostřed obrazovky a bude obsahovat název každého kroku a indikátor toho, který krok je aktuálně zobrazený. Po kliknutí na název kroku se uživatel přepne na tento krok. Toto řešení se mi líbí, protože je přehledné a uživatel je provázen celým procesem vytváření akce. Jediná nevýhoda tohoto řešení je, že zabírá hodně místa na obrazovce, což na mobilních zařízeních může být problém. Proto jsem pro mobilní zobrazení navrhla, aby se místo karty s kroky zobrazoval pouze název aktuálního kroku a progres bar, který bude indikovat, kolik kroků uživatel již prošel a kolik jich ještě zbývá. V obou případech se budou tlačítka na přechod mezi jednotlivými kroky nacházet v dolní části obrazovky. Na posledním kroku se místo tlačítka pro přechod na další krok bude nacházet tlačítko pro odeslání formuláře a vytvoření akce.

Dalším krokem bylo jak rozdělit informace do jednotlivých kroků. Jelikož je informací poměrně hodně i v rámci jednoho kroku, tak jsem se snažila informace rozdělit do logických celků, které spolu souvisí.

Dospěla jsem k následujícímu rozdělení:

- Krok 1 - základní informace o akci
  - Základní informace o akci (jméno akce, lokace)
  - Datum a čas konání akce - v případě jednorázové lze navíc doplnit datum konce akce, v případě opakované akce datum posledního výskytu akce a

frekvence opakování akce (denně, týdně, měsíčně)
  - Cena - kolik akce stojí a jaká je měna
- Krok 2 - popis akce - zvolila jsem proto tento prvek samostatný krok, protože z mých zkušeností bývá velmi dlouhý.
- Krok 3 - média týkající se akce (zvolit primární obrázek akce, který se bude zobrazovat v přehledu nadcházejících akcí, a další obrázky a videa týkající se akce)
- Krok 4 - doplňující informace o akci
  - Doplňující informace o akci (typ akce, typ tance, pro jakou úroveň tanečníků je akce určena, odkaz na facebookovou událost akce)
  - Informace ke kapacitě akce (maximální počet účastníků)

4.5.9 O aplikaci

Tato stránka nebyla potřeba nijak vzlášť navrhovat, protože bude obsahovat pouze textové informace o aplikaci, které budou rozděleny do několika odstavců. Stránka bude obsahovat informace z požadavku F27.

4.5.10 Stránka s registrovanými uživateli

Tato stránka bude přístupná pouze pro organizátory dané akce a bude obsahovat seznam všech registrací pro danou událost dle požadavku F15 a možnosti s registrací pracovat (například zrušit registraci, přijmout registraci, atd.). Možnosti přijímat nebo zamítat registrace jsou v souladu s požadavky F21 a F19.

Stránka bude tvořena názvem akce a pod ním bude datum akce. V případě, že je akce opakovaná, bude zde stejná komponenta jako u detailu akce, která dovolí přepínat mezi jednotlivými výskyty akce dle požadavku F26.

Pod tím se bude nacházet tabulka, která bude obsahovat informace o registracích pro danou akci. Hlavička tabulky bude obsahovat zaškrtávací pole pro označení všech registrací, informace o uživateli, který se registroval, datum registrace a status registrace. Při zaškrtnutí konkrétní registrace se zobrazí možnosti pro práci s touto registrací, například zrušení registrace, přijetí registrace, atd. Každá hlavička tabulky obsahuje vhodný filtr pro každý sloupec, což může umožnit například filtrovat registrace podle statusu registrace, který organizátor chce vidět.

4.5.11 Přihlašování a odhlašování z aplikace

Pro přihlašování do aplikace jsem navrhla samostatnou stránku, která obsahuje kartu s tlačítkem umožňujícím přihlášení pomocí účtu Google dle požadavku F3. V případě potřeby rozšíření o další způsoby přihlašování, například

pomocí e-mailu a hesla, by bylo možné tyto možnosti do stejné karty snadno doplnit.

Pokud se uživatel rozhodne z aplikace odhlásit, je po provedení této akce (dle požadavku F4) přesměrován na stránku s informací, že odhlášení proběhlo úspěšně. Tlačítko pro opětovné přihlášení jsem na tuto stránku záměrně neumísťovala, protože možnost přihlášení je již dostupná v navigační liště aplikace.

Kapitola 5
Implementace

V této kapitole je popsána implementace jednotlivých částí projektu (frontend, backend, databáze). Je popsáno, jak byly projekty vytvořeny a jaké knihovny/balíčky byly použity za účelem usnadnění vývoje. Dále jsou popsány konkrétní implementace klíčových funkcionalit a se kterými problémy jsem se při jejich implementaci setkala a jak jsem je řešila.

5.1 Server

Pro vytvoření backendové části projektu byl zvolen framework Spring Boot, přičemž samotný projekt byl založen v prostředí IntelliJ IDEA, které bylo určeno jako hlavní nástroj pro vývoj backendu. Toto vývojové prostředí automaticky vytvořilo základní strukturu projektu včetně konfigurace nástroje Gradle. IntelliJ IDEA zároveň poskytuje širokou škálu funkcionalit, které usnadňují vývoj, jako je přehledné ladění aplikace, integrace se systémem pro správu verzí Git či podpora práce s databázemi. [35]

5.1.1 Přihlášení uživatele

Přihlášení uživatele bylo realizováno pomocí Google na základě požadavku 3.1.1. Jelikož naše aplikace není jen klientská, ale i serverová, bylo potřeba implementovat přihlášení i na serveru.

Prvním krokem bylo vytvoření projektu v Google Cloud Console, Poté bylo potřeba vytvořit OAuth 2.0 Client ID pro webovou aplikaci, což nám poskytlo Client ID a Client Secret, které jsou nezbytné pro implementaci přihlášení. Následně byly nastaveny redirect URI, které jsou adresy, na které bude Google přesměrovávat uživatele po úspěšném přihlášení a origins, které jsou adresy, ze kterých bude možné přihlášení provést.

Poté byly přidány tzv. scopes, které určují, jaké informace o uživateli bude aplikace požadovat. V našem případě bude při prvotním přihlášení požadováno

pouze základní informace o uživateli, jako je jeho jméno a emailová adresa. Tudíž se jedná o následující scopes [36]:

- `openid` - Tento scope propojuje uživatele s jeho osobními údaji na Google účtu.
- `profile` - Tento scope umožňuje získat osobní údaje uživatele, i které zveřejnil
- `email` - Tento scope umožňuje získat primární emailovou adresu uživatele.

Tyto scope se řadí mezi nesenzitivní oprávnění, což jsou oprávnění, která slouží k autorizaci uživatele. [37]

Při nasazení aplikace do produkce a využívání api google, google nevyžaduje u těchto oprávnění ověření. Tudíž je jednodušší aplikaci uvést do produkce. [38]

5.1.1.1 Inkrementální autorizace

Aplikace bude kromě zmíněných oprávněních z předešlé sekce také vyžadovat oprávnění pro přístup k Google Forms API, aby uživatelé mohli propojit formulář a odpovědi z formuláře s aplikací. Oprávnění jsou následující [36]:

- `https://www.googleapis.com/auth/forms.body` - Tento scope umožňuje aplikaci číst a upravovat obsah formulářů, jako jsou otázky,
- `https://www.googleapis.com/auth/forms.responses.readonly` - Tento scope umožňuje aplikaci číst odpovědi z formulářů, ale neumožňuje je upravovat.

Jelikož se může stát že uživatel nebude nikdy potřebovat propojit formulář s aplikací, bylo by zbytečné požadovat tato oprávnění již při prvotním přihlášení. Proto je implementována tzv. inkrementální autorizace, která umožňuje požádat o další oprávnění až v momentě, kdy jsou potřeba. V našem případě se tedy o tato oprávnění požádá až v momentě, kdy se uživatel bude snažit propojit formulář s aplikací. Pokud uživatel tato oprávnění již udělil, nebude o ně požádán znovu. [39]

Obecně inkrementální autorizace funguje tak, že uživatel požádá o oprávnění, která potřebuje a endpoint Google vrátí kód, který dále může být vyměněn za access token obsahující všechny oprávnění, která uživatel udělil. Tento způsob je od Google považován jako doporučený, oproti například požádání všech oprávnění najednou. [39]

5.1.1.2 Proces přihlášení

Proces přihlášení začíná na straně klienta. Uživatel klikne na tlačítko Sign in a klient provolá api Google pro přihlášení.

Url, které klient volá pro přihlášení, vygeneruje funkce na základě parametrů, jejíž implementaci lze nahlédnout v kódu 5.1.

```
export const buildGoogleOAuthUrl = (params: {
  scope: string;
  state?: string;
}) => {
  const baseUrl = GOOGLE_OAUTH_URL;
  const urlParams = new URLSearchParams({
    client_id: GOOGLE_CLIENT_ID,
    redirect_uri: GOOGLE_REDIRECT_URI,
    response_type: 'code',
    scope: params.scope,
    access_type: 'offline',
    include_granted_scopes: true.toString()
  });

  if (params.state) {
    urlParams.append('state', params.state);
  }

  return `${baseUrl}?${urlParams.toString()}`;
};
```

Výpis kódu 5.1 Funkce pro generování url pro přihlášení přes Google OAuth

Funkci jsem si naprogramovala sama i přes to, že existuje například knihovna `@react-oauth/google`, která zjednodušuje proces přihlášení. Avšak při implementaci jsem narazila na problém, že tato knihovna v případě že využíváme flow autorization code, automaticky posílá parameter prompt=consent, což způsobuje, že uživatel musí při každém přihlášení proklikat souhlas s oprávněními, i když je již udělil. Tudíž jsem se rozhodla implementovat přihlášení sama, abych měla plnou kontrolu nad tím, jaké parametry jsou posílány a mohla tak využít inkrementální autorizaci.

Parametry, které jsou posílány v url pro přihlášení, jsou následující [40]:

- `client_id` - Toto je Client ID, které jsme získali při vytvoření OAuth 2.0 Client ID v Google Cloud Console.
- `redirect_uri` - Toto je URI, na které bude Google přesměrovávat uživatele po úspěšném přihlášení. Toto URI musí být stejné jako jedno z redirect URI, které jsme nastavili v Google Cloud Console.
- `response_type` - Tento parameter určuje, jaký typ odpovědi očekáváme od Google. V našem případě je to `code`, což znamená, že očekáváme autorizační kód, který následně vyměníme za access token. Druhým typem odpovědi je `token`, což znamená, že očekáváme přímo access token. Tento typ odpovědi se používá v tzv. implicitním flow, které se však nedoporučuje pro webové aplikace, protože není tak bezpečné jako authorization code flow.

- `scope` - Tento parameter obsahuje seznam oprávnění, o která žádáme. Oprávnění jsou oddělena mezery.
- `access_type` - Tento parameter určuje, zda chceme získat refresh token. Pokud je tento parameter nastaven na `offline`, Google nám pošle refresh token, který můžeme použít k získání nového access tokenu, když ten původní vyprší. Parametr jsem takto zvolila kvůli tomu, že bude vyžadováno abychom v budoucnu volali API i bez přítomnosti uživatele, například pro synchronizaci odpovědí z formuláře, a v takovém případě je potřeba mít možnost získat nový access token bez toho, aby se uživatel musel znovu přihlásit.
- `include_granted_scopes` - Tento parameter určuje, zda chceme, aby Google zahrnul již udělená oprávnění do access tokenu. Pokud je tento parameter nastaven na `true`, Google nám pošle access token, který obsahuje všechna oprávnění, která uživatel udělil, nejen ta, o která právě žádáme. Při implementaci se mi stalo, že jsem na tento parametr opomněla a v důsledku při každém přihlášení uživatel resetoval všechna oprávnění, která udělil, a musel je znovu udělovat, což samozřejmě nebylo ideální.
- `state` - Tento parametr je volitelný, ale doporučený, a slouží k ochraně proti CSRF útokům. Klient do něj vloží náhodný řetězec, který si uloží (např. do session). Po přesměrování zpět ho porovná s hodnotou v odpovědi. Tím ověří, že odpověď odpovídá požadavku iniciovanému stejným uživatelem, a ne cizímu požadavku.

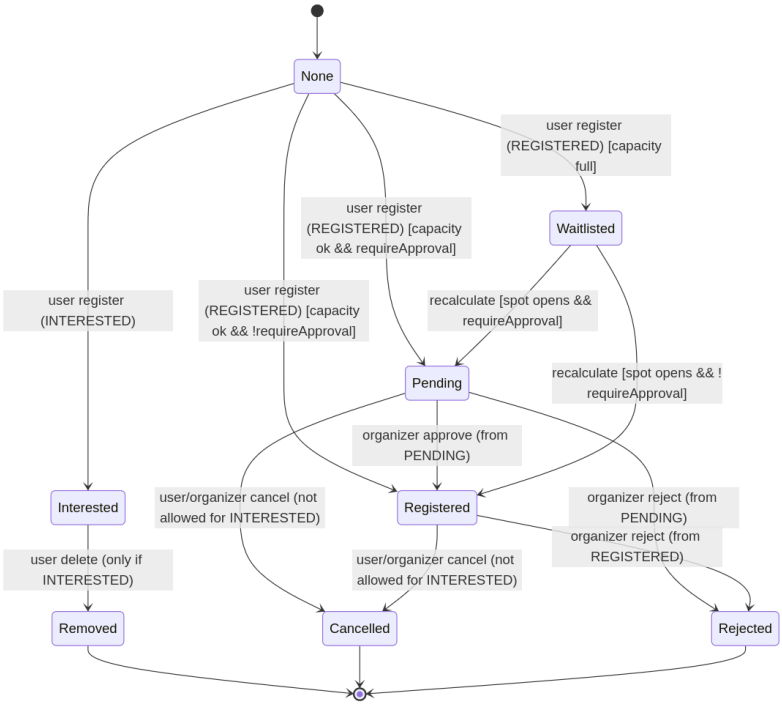
Potom co uživatel zvolí účet, kterým se chce přihlásit, a udělí požadovaná oprávnění (pokud je ještě neudělil), Google přesměruje uživatele na redirect URI, které jsme nastavili, a v URL bude obsažen autorizační kód. Klient tento kód zachytí a pošle ho na náš server, kde ho vyměníme za access token. Pokud se jedná o prvotní přihlášení, Google nám pošle i refresh token, který uložíme do databáze pro pozdější použití. Poté obratem klientovi vrátíme serverem vygenerovaný vlastní JWT token, který klient bude používat pro autentizaci a autorizaci při dalších požadavcích na server. [40]

Při testování přihlášení jsem narazila na problém, že když se přihlásím za uživatele po nějaké době, který měl již odsouhlasené práva na Google Forms API, nedostávám po nějaké době přístup k tomuto API, Google API mi vrací token obsahující pouze základní oprávnění. Po prouzkoumání jsem zjistila, že tím, že je aplikace v tzv. testovacím režimu, Google maže udělená oprávnění (kromě základních) po 7 dnech neaktivity uživatele. [41]

5.1.2 Registrace uživatele

Registrace uživatele začíná ve chvíli, když uživatel zaklikne v aplikace, že se chce registrovat anebo vyplní google formulář. V případě že se jedná o registraci přes aplikaci, uživatel v případě otevřené registrace jen potvrdí, že se chce

registrovat, a v případě partnerské registrace vybere roli, kterou chce zastávat. Po odeslání formuláře se vytvoří záznam v databázi s informací o registraci a jejím stavu (např. čekající na schválení, schválená, zamítnutá). V případě že se jedná o google formulář, je proces vysvětlen blíže v následující sekci. Organizátor akce pak může tyto registrace spravovat a měnit jejich stav. Stavový diagram pro registrace je zobrazen na obrázku 5.1 a popisuje jak se přiřazují stavy pro registraci a jaké akce jsou s těmito stavy spojeny.



Obrázek 5.1 Stavový diagram pro registrace.

V případě, že organizátor chce vidět všechny registrace a s nimi pracovat, bylo třeba implementovat endpoint, který vrací všechny registrace pro danou akci. Jelikož chceme aby implementace byla do budoucna rozšiřitelná, bylo třeba navrhnout strukturu odpovědi obecně, bez ohledu na to, zda se jedná o registrace z google formuláře nebo z aplikace. Proto jsem navrhla, že se budou zasílat hlavičky tabulky, a dále samotné registrace, ve kterých bude uloženo mapování mezi hlavičkou a hodnotou odpovědi. V případě, že se jedná o registraci přes aplikaci, máme staticky definované hlavičky (např. jméno, email, role), a v případě google formuláře jsou hlavičky dynamické a závisí na struktuře formuláře. Příklad odpovědi pro endpoint, který vrací registrace, je zobrazen v ukázce kódu 5.2.

```
1 {
2   "headers": [
3     {
```

```
4          "type": "TEXT",
5          "id": "16893cb4",
6          "question": "Zadejte jmeno a prijmeni"
7        },
8        {
9          "type": "SET",
10         "id": "2f58f63f",
11         "question": "Ktery kurz chcete",
12         "answerSet": [
13           "Sensual",
14           "blabla",
15           "salsa"
16         ]
17       },
18   ],
19   "registrations": [
20     {
21         "id": "62d8f653-cdd9-45de-9442-165d98102b0f",
22         "user": {
23           "email": "zanetatrosk@gmail.com",
24           "userId": "5e600195-ea89-4cdb-9011-c6235140675c"
25         },
26         "data": [
27           {
28             "id": "16893cb4",
29             "value": "Zaneta Troskova"
30           },
31           {
32             "id": "2f58f63f",
33             "value": "Sensual"
34           }
35         ],
36         "lastSubmittedTime": "2026-01-10T12:34:56Z",
37         "status": "REGISTERED"
38       },
39   ]
40 }
```

Výpis kódu 5.2 Příklad části struktury odpovědi pro endpoint vracející registrace

5.1.3 Integrace s Google Forms

Jelikož jedna z hlavních funkcí naší aplikace je možnost propojit formulář z Google Forms (viz v požadavku 3.1.1), bylo potřeba implementovat integraci s Google Forms API. Tato integrace nám umožní získávat strukturu formuláře a odpovědi, které uživatelé do formuláře zadali, a tyto informace následně zobrazovat v naší aplikaci.

Provolání samotného Google API je obdobné jako u autentizace, jen je

potřeba, aby uživatelova oprávnění obsahovala oprávnění pro přístup k Google Forms API, což jsem řešila pomocí inkrementální autorizace, jak jsem popsala v sekci 5.1.1.1, a dále je nutné znát ID formuláře. [42]

V rámci integrace provoláváme zejména dva endpointy Google Forms API [**google_forms_api**]:

- `GET /v1/forms/{formId}` – vrací formulář.
- `GET /v1/forms/{formId}/responses` – vrací seznam odpovědí odeslaných z formuláře.

V implementaci nejprve načítáme strukturu formuláře (`GET /v1/forms/{formId}`) a následně odpovědi (`GET /v1/forms/{formId}/responses`). Obě odpovědi jsou z Google API vráceny jako JSON, který následně mapujeme do interní struktury, kterou používáme v naší aplikaci.

1) Struktura odpovědi `GET /v1/forms/{formId}` (zjednodušeně)

```
{
  "formId": "1Qw8cABqYjtjOvValF28DpAEuZ2q4jwerABQ_v2jtYcc",
  "revisionId": "...",
  "info": {
    "documentTitle": "testik"
  },
  "items": [
    {
      "itemId": "0c9e2e5a",
      "questionItem": {
        "question": {
          "questionId": "id1",
          "required": true,
          "textQuestion": {}
        }
      },
      "title": "Zadejte jmeno a prijmeni"
    }
  ]
}
```

Z této odpovědi zpracováváme zejména [43]:

- `revisionId` pro detekci změny formuláře,
- `items[*].questionItem.question.questionId` interní identifikátor otázky,
- `items[*].title` text hlavičky/sloupce,
- typ otázky (`textQuestion`, `choiceQuestion`, `dateQuestion`, ...) podle něj volíme typ interní hlavičky.

2) Struktura odpovědi `GET /v1/forms/{formId}/responses` (zjednodušeně)

```
{
  "responses": [
    {
      "responseId": "...",
      "respondentEmail": "user@example.com",
      "lastSubmittedTime": "2026-01-10T12:34:56Z",
      "createTime": "2025-11-26T20:25:39.414Z",
      "answers": {
        "id1": {
          "questionId": "id1",
          "textAnswers": {
            "answers": [
              {
                "value": "..."
              }
            ]
          }
        }
      },
    }
  ]
}
```

Z odpovědí zpracováváme zejména [44]:

- `responseId` unikátní identifikátor odpovědi (ukládán k registraci),
- `respondentEmail` email respondenta (pokud je dostupný),
- `lastSubmittedTime` čas posledního odeslání (pro detekci aktualizace),
- `answers` mapování `questionId` → hodnota odpovědi.

Po úspěšném napojení na Google Forms API jsem nejprve naivně navrhla že pokaždé co organizátor navštíví stránku a provolá GET Registrations tak se provolá Google API. Toto řešení sice zaručovalo, že má organizátor up-to-date strukturu formuláře a odpovědi, avšak pokaždé se musel provolat HTTP požadavek a dojít k parsování google odpovědi na strukturu, která je v API. To zapříčinilo, že odpověď trvala déle (naměřila jsem 2 sekundy v prohlížeči). Další důvod proč jsem začala optimalizovat, je ten že google má určité kvóty na provolávání API. Tudíž se vyplatí požadavkami šetřit. Rozhodla jsem se tedy pro způsob že strukturu formuláře a odpovědi zachachuji. Jelikož jedna odpověď Google formuláře je jedna registrace v databázi, tak v tomto případě si budeme držet responseId odpovědi a dále celou odpověď. Podobným způsobem si budu

ukládat i strukturu formuláře v tabulce `registrations_settings`. Oboje je uloženo v databázi ve formátu JSON pomocí datového typu JSONB pro efektivnější načítání a parsování [45]. Po této naimplementované optimalizaci se doba požadavku změnila na 300 ms. Avšak stále zůstal problém kdy formulář synchronizovat. Rozhodla jsem se, že naimplementuji job, který se bude spouštět za stanovený čas(já jsem zvolila jednou za hodinu) a synchronizovat na pozadí jak formulář tak i odpovědi z něj. Synchronizovat mohu i přes to, že je uživatel neaktivní, protože mám uložený refresh token, který mi umožňuje získat nový access token a provolat Google API.

Pokud zjistím, že je příliš požadavků na stanovenou kvótu, můžu dobu spouštění upravit. Problém avšak bude nastávat v situacích kdy organizátor potřebuje synchronizovat registrace ihned (např. pokud akce začíná a je možné že se někdo přihlásil v časovém okně, kde ještě nedošlo k synchronizaci). Pro takové případy jsem se rozhodla naimplementovat endpoint, který umožní synchronizaci spustit ihned. Tato změna se projeví i v uživatelském rozhraní kde bude implementováno tlačítko, které endpoint provolá.

Při synchronizaci formuláře kontroluji zda se změnila struktura formuláře, což zjišťuji porovnáním `revisionId` z Google API a `revisionId` uloženého v databázi. Pokud se `revisionId` liší, znamená to, že došlo ke změně formuláře a je potřeba aktualizovat strukturu formuláře v databázi. Pokud se `revisionId` neliší, nemusím aktualizovat strukturu formuláře, protože vím že se nezměnila. Podobným způsobem kontroluji i odpovědi, kde porovnávám čas posledního odeslání odpovědi z Google API a čas poslední synchronizace odpovědi v databázi.

5.2 Frontend

Jak už bylo řečeno v sekci Návrh, frontend část projektu bude napsána ve frameworku React. Pro založení projektu jsem se rozhodla použít Tanstack Start, což je full-stack React framework založený na dvou klíčových technologiích: Tanstack Router pro routování a Vite pro build projektu. [46]

Tanstack Start nabízí několik výhod, jako je například rychlý vývoj a flexibilní nasazení kombatibilní s Vite, jednoduché routování díky Tanstack Router, zabudované nástroje pro vývojáře a také integrace s knihovnami pro efektivní načítaní dat z API (Tanstack Query, Apollo). [47]

5.2.1 API na lokace

Při tvorbě taneční akce je potřeba zadat lokaci, kde se bude konat. Jelikož nechci aby uživatelé zadávali lokaci ručně (je to nepohodlné a může to vést k chybám), bylo potřeba najít správné API, které umožňuje získávat lokace zadané uživatelem. Zároveň ale pořád chci umožnit lokaci ručně upravit, protože je možné, že některé lokace budou v API chybět (například se bude v API nacházet ulice, ale ne s číslem popisným, na které se akce bude konat). V

následujících odstavcích popíšu, mezi jakými API jsem vybírala a která byla nakonec zvolena.

5.2.1.1 Google places API (New)

Jedná se o vystavenou službu od společnosti Google, která umožňuje pracovat s lokacemi. Toto API nabízí širokou škálu funkcí [48] jako například:

- Place Details (New) - API umožňuje získat podrobné informace o konkrétním místě, jako je adresa, telefonní číslo, hodnocení, fotografie a další.
- Text Search (New) - API umožňuje hledat místa na základě textového dotazu, jako je název místa, adresa nebo kategorie.
- Nearby Search (New) - API umožňuje hledat místa v okolí zadaných geografických souřadnic.
- Autocomplete (New) - API umožňuje získat návrhy míst na základě částečného textového dotazu a geografického rozsahu.

S tímto API jsem se již setkala v několika aplikacích při zadávání lokace pro události (například v aplikaci OnlyDance).

Toto API jsem nakonec nezvolila, kvůli tomu, že je placené a cena se odvíjí od počtu požadavků, které jsou na API posílány. Jelikož nechci aby projekt se stal finančně náročným, rozhodla jsem podívat se po nějaké levnější alternativě, která by mi poskytla podobné funkce.[49]

5.2.1.2 Photon

Toto API je open-source a je založeno na OpenStreetMap datech. API nabízí funkce jako například:

- Geocoding
- Reverse Geocoding
- hledání míst - podporuje i v rámci několika jazyků

Tento projekt má i vystavené zdarma API, které je možné využívat bez nutnosti hostovat vlastní instanci. Avšak je třeba dbát na to nezahlcovat toto API přílišným množstvím požadavků, protože na to server není připraven a jedná se spíše o ukázku toho, jak API funguje. Pokud by bylo třeba posílat větší množství požadavků, tak je v projektu zdokumentováno jak hostovat vlastní instanci. [50]

Nakonec jsem se rozhodla pro využití tohoto API, protože je zdarma a poskytuje všechny funkce, které potřebuji pro zadávání lokace pro události. Jelikož moje aplikace je ve fázi prototypu, nebude na API posíláno příliš mnoho požadavků, takže by neměl být problém s využíváním tohoto API. V budoucnu,

pokud by se aplikace stala populární a bylo by potřeba posílat větší množství požadavků, bych zvážila hostování vlastní instance tohoto API nebo přechod na placené API, jako je Google Places API. Navíc při zadávání lokace bude nastavený přiměřený *debounce*, aby se zabránilo posílání příliš mnoha požadavků na API.

5.2.2 Formuláře

Aplikace obsahuje několik formulářů, ať už se jedná o formulář pro vytvoření akce, nastavení registrace, úprava profilu a další. U jednoduchých formulářů jako je úprava profilu, stačilo stav formuláře řešit pomocí useState, ale u složitějších formulářů jako je například formulář pro vytvoření akce, jsem chtěla použít knihovnu, která by mi usnadnila práci s formulářem a zároveň by mi poskytla funkce jako například validace.

Po prozkoumání několika knihoven jsem se rozhodla pro Tanstack Form, protože mám dobré zkušenosti s Tanstackem jako takovým a tato knihovna nabízí potřebné funkce pro práci s formuláři jako validace, error handling, správa stavu formuláře a další. Navíc je tato knihovna dobře zdokumentovaná, což usnadňuje její použití. [51]

5.2.3 Načítání dat z API

Pro načítaní dat z API jsem se rozhodla použít knihovnu Tanstack Query, která poskytuje funkce pro efektivní načítaní dat z API, jako je například cachování, synchronizace dat, stránkování a další. Tato knihovna mi umožňuje jednoduše načítat data z API a zároveň se stará o další aspekty načítaní dat, jako je například zobrazení loading stavu, error stavu a další. [52]

V kódu 5.3 je ukázka toho, jak načítám data z API pomocí Tanstack Query v routě pro profil uživatele.

5.2.4 Routování

Tím, že jsme vytvořili projekt pomocí TanstackStart, máme již předpřipravenou funckionalitu pro routování v aplikaci, protože Tanstack Start zahrnuje Tanstack Router. Tato knihovna používá tzv. file-based routing, což znamená, že struktura souborů v adresáři `src/routes` určuje strukturu URL v aplikaci. [53]

Například soubor `src/routes/my-profile.tsx` bude odpovídat URL `/my-profile`. V ukázce kódu 5.3 lze vidět definovanou routu, kde v případě že uživatel není přihlášen, bude přesměrován na přihlašovací stránku, a pokud je přihlášen, načtou se data z API a zobrazí se profilová stránka s těmito daty.

```
1 export const Route = createFileRoute("/my-profile")({
2   beforeLoad: requireAuth,
3   component: RouteComponent,
```

```
4  });
5
6  function RouteComponent() {
7    const { user } = useUser();
8    const { data } = useQuery({
9      queryKey: ["profile", user?.userId],
10     queryFn: () => getUserById(user?.userId),
11   })
12
13   if (!user || !data) {
14     return null;
15   }
16   return <ProfilePage userId={user.userId}
       profileDataDefault={data} key={user.userId} />;
17 }
```

Výpis kódu 5.3 Příklad definice routy pro profil uživatele

5.3 Databáze

Jak již bylo zmíněno v sekci Návrh, pro databázi jsem zvolila relační databázi PostgreSQL. Při implementace databáze jsem vytvořila dva skripty pro vytvoření tabulek a vložení testovacích dat. První skript `create_tables.sql` obsahuje SQL příkazy pro vytvoření všech tabulek podle schématu navrženého v sekci Návrh. Druhý skript `insert_test_data.sql` obsahuje SQL příkazy pro vložení testovacích dat do tabulek.

Pro média jsem se rozhodla využít objektové úložiště MinIO, jak jsem popsala v sekci Návrh. Při ukládání médií uložím do databáze pouze URL, které odkazuje na dané médium v objektové databázi. Samotné médium je však uloženo v objektové databázi.

Kapitola 6

Testování

Tato kapitola popisuje testování aplikace. Nejprve jsou zadefinovány pojmy týkající se testování. Poté je uvedeno, jakým způsobem bylo testování funkcionalit systému prováděno a proč byl zvolen právě tento způsob testování. Závěrečná část kapitoly se věnuje uživatelskému testování, jež bylo prováděno s potenciálními uživateli aplikace. Na konci kapitoly jsou shrnuty výsledky testování.

6.1 Testování

Testování je proces, při kterém se ověřuje a verifikuje, zda software splňuje požadavky a funguje správně. Primárním přínosem testování je doručení kvalitního softwaru díky včasnému odhalení chyb a zlepšení výkonu. V dnešním době je testování běžně součástí vývoje a již nebývá finální krokem před vydáním softwaru, ale je prováděno průběžně během vývoje. [54]

Testy bývají rozdělovány primárně do dvou kategorií [54]:

- **Manuální testování** – testování, které je prováděno testerem ručně bez použití automatizovaných nástrojů. Testeři ručně simulují klikání tlačítek, vyplňování textu a tím simulují chování koncového uživatele. Z mojí zkušenosti se manuální testování používá pro finální ověření nové funkcionality, před vydáním nové verze aplikace anebo testování částí systému, kde je obtížné vytvořit automatizované testy.

- **Automatizované testování** – testování, které je prováděno pomocí nástrojů a skriptů, které automaticky spouštějí testy na daném softwaru. Automatizované testy bývají rychlejší a jsou vhodné zejména pro repetivní testovací scénaře. Umožňují snížit riziko lidské chyby a konzistentně testovat systém.

6.1.1 Typy testů

V následující sekci uvedu typy testů a krátce popíšu, co každý z nich znamená. Existuje mnoho různých typů testů, ale pro účely této práce se zaměřím na ty nejběžnější, které jsou relevantní pro testování webových aplikací. Mezi tyto typy testů [55] patří:

- **Unit testy** – testy, které se zaměřují na testování jednotlivých částí kódu za cílem ověření, že samostatné části systému fungují správně jak mají. Díky tomu se testování zrychluje a omezuje se spouštění testů, které by jinak mohly být zbytečné.
- **Integration testy** – testy, které se zaobírají na testování rozhraní obsahující více částí aplikací, modelů nebo komponent za cílem ověřit, že systém funguje správně jako celek [56].
- **End-to-end testy** – testy, které ověřují, že proces v aplikaci od začátku do konce funguje správně.
- **Smoke Testing** – testy, které validující stabilitu systému. Bývají spouštěny na prvotní verzi aplikace pro kontrolu, zda systém obsahuje základní funkcionality.
- **Acceptance Testing** – testy, které bývají prováděny stakeholdery či koncovými uživateli za účelem ověření, že systém splňuje požadavky a je připraven k nasazení.

6.1.2 Způsob testování

Pro testování funkcionalit systému jsem zvolila kombinaci manuálního a automatizovaného testování. V případě unit testů bylo zvoleno automatizované testování, protože umožňuje rychlé a efektivní testování jednotlivých částí kódu. Pro integration testy a end-to-end testy jsem zvolila manuální testování, protože tyto testy často vyžadují komplexní scénáře a interakce mezi různými částmi systému, které mohou být obtížné automatizovat. Při přidání nové funckcionality provádím vždy manuální smoke testy, abych ověřila, že základní funkcionality systému fungují správně. Před vydáním prototypu aplikace jsem se rozhodla provést acceptance testy s potencionálními uživateli, abych získala zpětnou vazbu a ověřila, že systém splňuje požadavky a je připraven k budoucímu nasazení.

6.2 Implementace testů pro jednotlivé části systému

V následujících podsekcích popíši implementaci automatizovaných unit testů pro frontend a backend. Vysvětlím, jaké nástroje jsem použila pro testování a jak jsem postupovala při psaní testů pro jednotlivé části systému.

6.2.1 Frontend

Pro frontend unit testing jsem použila testovací framework Vitest [57], který je součástí ekosystému TanStack. Psaní testů usnadňovalo fakt, že jsem měla zkušenosti s podobným testovacím frameworkem zvaným Jest.

Při psaní unit testů pro frontend jsem se zaměřila na testování především částí obsahující logiku, většinou jsou tyto funkce umístěné ve složkách utils. V kódu 6.1 je možné vidět příklad unit testu pro funkci, která zobrazuje správný text na základě ceny akce z backendu. Test kontroluje, že pro akci s cenou 0 Kč se zobrazí text "Free", pro akci s cenou 100 Kč se zobrazí text "100 Kč"a pro akci s cenou null se zobrazí text "Not specified".

```
import { describe, expect, it } from 'vitest'
import { getLabelFromPrice } from './getLabelFromPrice'

describe('getLabelFromPrice', () => {
  it('returns "Not specified" when price is undefined', ()
      => {
    expect(getLabelFromPrice(undefined, 'USD')).toBe('Not
        specified')
  })

  it('returns "Not specified" when currency is missing', ()
      => {
    expect(getLabelFromPrice(10, undefined)).toBe('Not
        specified')
  })

  it('returns "Free" when price is 0 and currency is present
      ', () => {
    expect(getLabelFromPrice(0, 'CZK')).toBe('Free')
  })

  it('formats a non-zero price with two decimals', () => {
    expect(getLabelFromPrice(12.5, 'EUR')).toBe('12.50 EUR')
    expect(getLabelFromPrice(12, 'EUR')).toBe('12.00 EUR')
  })
})
```

Výpis kódu 6.1 Příklad unit testu pro frontend

6.2.2 Backend

Na unit testování backendu jsem vzhledem k tomu, že je backend psaný ve frameworku Spring boot zvolila testovací framework JUnit, se kterým jsem se setkala standartně v rámci backend vývoje v Spring boot. [58]

6.3 Uživatelské testování

Uživatelské testování bylo prováděno s potencionálními uživateli aplikace, kteří byli vybráni na základě jejich zájmu o danou problematiku. Uživatelské testování probíhalo formou individuálních sezení, během kterých uživatelé prováděli různé úkoly v aplikaci, zatímco já jsem sledovala jejich interakce a sbírala zpětnou vazbu. Na konci těchto sezení jsem kladla otázky týkající se jejich zkušeností s aplikací, co se jim líbilo a co by mohlo být zlepšeno.

Před zahájením samotného testování byly navrženy scénáře úloh, které měli účastníci během testování plnit. Scénáře byly rozděleny do dvou skupin na scénáře pro běžného uživatele (tanečníka) a scénáře pro organizátora akcí. Snažila jsem se scénáře navrhnout tak, aby otestovaly klíčové funkce aplikace z pohledu obou typů uživatelů, a zároveň aby byly dostatečně jednoduché na pochopení a nebyly příliš časově náročné.

6.3.1 Scénáře pro běžného uživatele

Tyto scénáře byly navrženy tak, aby otestovaly různé funkce aplikace z pohledu běžného uživatele, který je tanečníkem, ale neorganizuje akce.

Přihlášení do aplikace a práce s profilem

1. Klikněte v navigaci na tlačítko "Sign in"a přihlaste se pomocí Google účtu pod emailem zanetrosk@gmail.com.
2. Přejděte na stránku profilu.
3. Klikněte na tlačítko "Edit profile"a upravte informace dle gusta. Uložte změny a zkontrolujte, zda se změny projevily v profilu.

Vyhledávání a operace s tanečními akcemi

1. Přejděte na stránku přehledu tanečních akcí.
2. Vyfiltrujte akce, které se konají v Praze, obsahují tanec "salsa"a jsou to party.
3. Vyberte akci, která vás zajímá a zaklikněte, že máte o ni zájem.
4. Přejděte na stránku s vašimi akcemi a zkontrolujte, zda akce, kterou jste zaklikli, že máte o ni zájem, je zobrazena mezi akcemi, které vás zajímají.
5. Přejděte z této stránky na detail této akce

Práce s detailem akce a přihlášení na akci

1. Přejděte na detail akce "Bachata hit night".
2. Rozklikněte profil pořadatele akce.
3. Vraťte se zpět na detail akce a přihlaste se na akci neanonymizovaně.
4. Podívejte se na seznam přihlášených účastníků a zkontrolujte, zda je vaše jméno zobrazeno mezi přihlášenými.
5. Rozklikněte místo konání akce.
6. Rozklikněte facebook událost akce.
7. Zkontrolujte, že se akce zobrazuje v přehledu vašich akcí, na které jste přihlášeni.
8. Zrušte přihlášení na akci.

Přihlášení na akci prostřednictvím google formuláře

1. Přejděte na detail akce "Zouk practice".
2. Přihlašte se na akci prostřednictvím google formuláře.
3. Po čase zkontrolujte, že jste přihlášeni na akci.

6.3.2 Scénáře pro organizátora akcí

Tyto scénáře byly navrženy tak, aby otestovaly funkce aplikace z pohledu organizátora tanečních akcí. Ve druhém scénáři se uživatelé přihlašují pod existujícím účtem, a to z důvodu přípravy testovacích dat. Pro otestování tohoto scénáře je totiž nutné mít předem připravený Google formulář obsahující odpovědi od uživatelů.

Přihlášení do aplikace, operace s akcí z pohledu organizátora

1. Klikněte v navigaci na tlačítko "Sign in"a přihlaste se pomocí Google účtu.
2. Klikněte na tlačítko "Create event"v navigaci.
3. Vyplňte formulář pro založení akce s názvem "Testovací akce", která se bude konat v Praze, bude určena pro pokročilé tanečníky a bude obsahovat tanec "salsa". Uložte akci.
4. Zkontrolujte zda je akce "Testovací akce"zobrazena v přehledu vašich akcí a přejděte na její detail.

5. Publikujte akci jako párovou a zkontrolujte, že je akce zobrazena v přehledu všech akcí.
6. Zrušte publikaci akce a zkontrolujte, že je akce označena jako zrušená v přehledu všech akcí.

Založení opakované akce, publikace pomocí google formuláře a práce s registracemi

1. Přihlaste se pomocí Google účtu za profil "Anna Nová".
2. Vytvořte opakovanou akci s názvem "Testovací opakovaná akce", která se bude konat každé pondělí v Praze po 4 týdny
3. Publikujte první výskyt akce pomocí google formuláře pomocí poskytnutého ID a zkontrolujte, že je akce zobrazena v přehledu všech akcí. Při publikaci zaškrtněte, že chcete potvrzovat registrace ručně.
4. Přejděte na detail akce "Testovací opakovaná akce"na jeho první výskyt.
5. Přejděte na seznam registrací pro tento výskyt.
6. Zrušte registraci tanečnice "Justyna Kowalska"pro tento výskyt akce.
7. Přejděte na profil tanečníka pod jménem "Zanda 2".
8. Vraťte se zpět na seznam registrací pro tento výskyt akce.
9. Potvrďte registraci tanečníka "Zanda 2"pro tento výskyt akce.
10. Synchronizujte registrace.

6.3.3 Uživatelé účastnící se testování

Pro uživatelské testování jsem oslovila 4 potencionální uživatele aplikace. Při výběru uživatelů jsem se zaměřila na různé věkové skupiny a taneční úrovně, abych získala co nejširší spektrum zpětné vazby. Uživatele nejmenuji, protože chci zachovat jejich anonymitu. Persony uživatelů, které se zůčastnili testování byli následující:

- **Person A** – 31 let, pokročilý tanečník, účastní se tanečních akcí, ale neorganizuje je.
- **Person B** – 23 let, začínající tanečnice, hledá taneční akce pro začátečníky.
- **Person C** – 30 let, středně pokročilý tanečník, plánuje organizovat taneční akce.
- **Person D** – 30 let, středně pokročilý tanečník, organizuje čtvrteční taneční akce.

6.3.4 Výsledky testování

Během uživatelského testování byla získána cenná zpětná vazba od všech zúčastněných respondentů. Většina uživatelů pozitivně hodnotila jednoduchost a přehlednost uživatelského rozhraní a neměla potíže s orientací v aplikaci. V průběhu testování však byly identifikovány i dílčí nedostatky, které byly následně odstraněny.

Jedním z problémů byla vizuální nejednoznačnost karet událostí – při najetí kurzoru se zobrazovala animace stínu, která evokovala interaktivitu, ačkoliv karta samotná nebyla klikací. Tento nedostatek byl vyřešen odstraněním dané animace. Další nejasnost se týkala tlačítka pro zobrazení detailu akce, které obsahovalo ikonu šipky naznačující otevření odkazu v nové záložce, přestože se detail zobrazoval ve stejném okně. Pro zvýšení konzistence uživatelského rozhraní byla tato ikona odstraněna.

Uživatelé rovněž upozornili na nedostatek v implementaci filtrování událostí, které nebylo řešeno pomocí query parametrů. V důsledku toho docházelo při použití navigačního tlačítka „zpět“ v prohlížeči ke ztrátě nastavených filtrů. Tento problém byl odstraněn zavedením filtrování prostřednictvím query parametrů, čímž byla zajištěna perzistence uživatelského nastavení.

V neposlední řadě byl upraven mechanismus vyhledávání lokací. Původní implementace zobrazovala informaci o nenalezení výsledků i v situaci, kdy samotné vyhledávání ještě neproběhlo. K tomu docházelo z důvodu, že frontend zobrazoval tuto zprávu již při změně hodnoty ve vstupním poli, pokud byl současně počet zobrazených lokací roven nule. Tato logika byla upravena tak, aby odpovídala očekávanému chování. Informace o nenalezení výsledků se nyní zobrazuje pouze v případě, kdy uživatel zadá vyhledávací dotaz a následně proběhne vyhledávání, které skutečně nevrátí žádné výsledky.

Kapitola 7

Závěr

Cílem této diplomové práce byl návrh a implementace webové aplikace pro správu tanečních akcí a kurzů v oblasti společenských tanců, především pro bachatu, salsu a zouk. Nejprve jsem analyzovala současné způsoby, jak tanečníci a organizátoři vyhledávají, propagují a spravují události, a na základě zjištěných problémů jsem společně s konzultacemi v taneční komunitě stanovila požadavky, na kterých bude systém stavět.

Na tyto požadavky navázal návrh architektury, databázového modelu a uživatelského rozhraní. Následně jsem navržené řešení implementovala jako klient–server aplikaci s frontendem v Reactu a backendem ve Spring Bootu (Kotlin), s využitím PostgreSQL pro perzistenci dat a MinIO pro ukládání médií.

V průběhu vývoje jsem prováděla manuální ověřování funkcionalit a klíčové části logiky jsem pokryla automatizovanými unit testy na frontendu i backendu. Ke konci práce jsem realizovala uživatelské testování s pěti respondenty z cílové skupiny, které potvrdilo srozumitelnost rozhraní a zároveň přineslo konkrétní podněty pro vylepšení, jež byly následně implementovány.

Práce mi přinesla také nové praktické zkušenosti. Na backendu jsem si osvojila vývoj v jazyce Kotlin ve Spring Boot a na frontendu jsem se naučila pracovat s ekosystémem TanStack. Pro tvorbu uživatelského rozhraní jsem získala zkušenosti s Tailwind CSS a komponentami `shadcn/ui`.

Stanovené cíle práce jsem splnila a výsledkem je použitelný prototyp aplikace zahrnující veškeré funkcionality vycházející z požadavků kategorií Must have a Should have. Uživatelům umožňuje efektivně vyhledávat taneční akce, filtrovat je podle zvolených kritérií, přihlašovat se na jednotlivé události a spravovat svůj uživatelský profil. Organizátorům pak poskytuje nástroje pro tvorbu a správu akcí, včetně možnosti jejich publikace různými způsoby, jako je registrace párů, otevřená registrace nebo prostřednictvím Google formuláře.

Během uživatelského testování se potvrdilo, že aplikace má potenciál stát se užitečným nástrojem pro taneční komunitu. Současně byly zohledněny a implementovány všechny relevantní připomínky uživatelů.

Do budoucna je plánováno pokračování ve vývoji aplikace, zejména rozšiřování funkcionalit na základě požadavků kategorií Could have a Won't have. Dále je v plánu realizace dalšího kola uživatelského testování s širší skupinou respondentů za účelem získání podrobnější zpětné vazby. V neposlední řadě se počítá s nasazením aplikace do produkčního prostředí, následným monitorováním jejího výkonu a spokojenosti uživatelů, což umožní pružně reagovat na vzniklé problémy a dále rozvíjet systém podle aktuálních potřeb.

Příloha A
Nějaká příloha

Sem přijde to, co nepatří do hlavní části.

Bibliografie

1. JETTE. *Social Dance Definition and Examples* [online]. 2024 [cit. 2025-11-27]. Dostupné z: https://www.jettence.com/blog/social-dance-definition-and-examples/.
2. PIZAZZ, Dance. *Salsa* [online] [cit. 2025-11-27]. Dostupné z: https://dance-pizazz.com/salsa/.
3. SALSA, Bohemian. *Historie salsy* [online] [cit. 2025-11-27]. Dostupné z: https://www.bohemiansalsa.cz/kurzy/co-ucime/historie-salsy/.
4. ALTADANCE. *Salsa Rueda de Casino* [online]. 2020 [cit. 2025-11-27]. Dostupné z: https://www.altadance.cz/salsa-rueda-de-casino/.
5. SBK90. *Danses* [online] [cit. 2026-03-10]. Dostupné z: https://www.sbk90.fr/danses/.
6. INCOGNITO DANCE. *What is Bachata?* [Online]. 2012 [cit. 2026-03-04]. Dostupné z: https://www.incognitodance.com/what-is-bachata-2/.
7. JT DANCE STUDIO. *Types of Bachata Dancing Styles* [online]. 2025 [cit. 2026-03-04]. Dostupné z: https://jtdancestudio.com/types-of-bachata-dancing-styles/.
8. ZOUK MINNEAPOLIS. *What is Brazilian Zouk?* [Online]. 2026 [cit. 2026-03-05]. Dostupné z: https://www.zoukminneapolis.com/what-is-brazilian-zouk.
9. CENTRUM TANCE. *Dance fusion* [online]. 2026 [cit. 2026-03-04]. Dostupné z: https://www.centrumtance.cz/styl/17/street-a-shuffle-dance/316/dance-fusion.
10. PRODUCTPLAN. *MoSCoW Prioritization* [online]. 2025 [cit. 2025-02-16]. Dostupné z: https://www.productplan.com/glossary/moscow-prioritization/.

11. STEINER, Elizabeth. *Non-functional vs. functional requirements: When to use each type* [online]. 2025 [cit. 2025-02-16]. Dostupné z: https://specinnovations.com/blog/non-functional-vs.-functional-requirements-when-to-use-each-type.
12. IAMPROVIDENCE. *MVC vs MVP vs MVVM with C# examples* [online]. 2022 [cit. 2026-03-12]. Dostupné z: https://medium.com/@iamprovidence/mvc-vs-mvp-vs-mvvm-with-c-examples-8013745e3c4c.
13. POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL: The World's Most Advanced Open Source Relational Database* [online]. 2026 [cit. 2026-03-12]. Dostupné z: https://www.postgresql.org/.
14. TENCENT CLOUD. *What database should be used to store audio files?* [Online]. 2026 [cit. 2026-03-17]. Dostupné z: https://www.tencentcloud.com/techpedia/132923.
15. MINIO, INC. *minio: High-performance, S3 compatible object store* [online]. 2026 [cit. 2026-03-17]. Dostupné z: https://github.com/minio/minio.
16. NESTJS. *NestJS Documentation* [online]. 2026 [cit. 2026-03-08]. Dostupné z: https://docs.nestjs.com/.
17. NESTJS. *Database* [online]. 2026 [cit. 2026-03-12]. Dostupné z: https://docs.nestjs.com/techniques/database.
18. SPENCER BOS. *Java Basics: What Is Spring Boot?* [Online]. 2025 [cit. 2026-03-12]. Dostupné z: https://www.jrebel.com/blog/what-is-spring-boot.
19. IBM. *What is Java Spring Boot?* [Online] [cit. 2026-03-12]. Dostupné z: https://www.ibm.com/think/topics/java-spring-boot.
20. VMWARE, INC. *Why Spring* [online]. 2026 [cit. 2026-03-12]. Dostupné z: https://spring.io/why-spring.
21. FARAKATE, Roshan. *MVC architecture in Spring Boot projects* [online]. 2024 [cit. 2026-03-12]. Dostupné z: https://medium.com/@roshanfarakate/mvc-architecture-in-spring-boot-project-14dadefe2c77.
22. SCAND. *Pros and Cons of Using Spring Boot* [online]. 2026 [cit. 2026-03-12]. Dostupné z: https://scand.com/company/blog/pros-and-cons-of-using-spring-boot/.
23. VIGNARAJJ. *Spring Boot with Java vs Spring Boot with Kotlin — A Comprehensive Comparison* [online]. 2021 [cit. 2026-03-12]. Dostupné z: https://medium.com/@vignarajj/spring-boot-with-java-vs-spring-boot-with-kotlin-a-comprehensive-comparison-83f4f8350066.
24. SHLOKA. *What The Heck Is a Single Page Application Anyway?* [Online]. 2025 [cit. 2026-03-12]. Dostupné z: https://dev.to/shlokaguptaa/what-the-heck-is-a-single-page-application-anyway-47op.

25. DRC SYSTEMS. *Guide to Single Page Applications* [online]. 2024 [cit. 2026-03-12]. Dostupné z: https://www.drcsystems.com/blogs/guide-to-single-page-applications/.

26. META PLATFORMS, INC. *React Documentation* [online] [cit. 2026-03-12]. Dostupné z: https://react.dev/.

27. REES, Damian. *What is wireframing* [online]. 2024 [cit. 2025-02-22]. Dostupné z: https://www.experienceux.co.uk/faqs/what-is-wireframing/.

28. FIGMA. *What is wireframing* [online] [cit. 2025-02-22]. Dostupné z: https://www.figma.com/resource-library/what-is-wireframing/.

29. DANIELYAN, Daniel. *10 reasons for UX designers to use Figma* [online]. 2021 [cit. 2025-02-23]. Dostupné z: https://medium.com/design-bootcamp/10-reasons-for-ux-designers-to-use-figma-981e9efd9f13.

30. LEARN, Figma. *Start designing with UI kits* [online] [cit. 2026-02-23]. Dostupné z: https://help.figma.com/hc/en-us/articles/24037724065943-Start-designing-with-UI-kits.

31. BERGMAN, Chad. *Design system 101: What is a design system?* [Figma Blog]. 2024 [cit. 2026-02-23]. Dostupné z: https://www.figma.com/blog/design-systems-101-what-is-a-design-system/.

32. GOPINATH, Vishwas. *15 Best React UI Libraries for 2026* [online]. 2025-12 [cit. 2026-02-24]. Dostupné z: https://www.builder.io/blog/react-component-libraries-2026.

33. *Tailwind CSS* [online]. 2025 [cit. 2026-02-24]. Dostupné z: https://www.builder.io/glossary/tailwind-css.

34. SHADCN. *Theming* [online] [cit. 2026-03-15]. Dostupné z: https://ui.shadcn.com/docs/theming.

35. JETBRAINS. *IntelliJ IDEA Features* [online]. 2026 [cit. 2026-03-26]. Dostupné z: https://www.jetbrains.com/idea/features/.

36. GOOGLE. *OAuth 2.0 Scopes for Google APIs* [online]. 2024 [cit. 2025-03-01]. Dostupné z: https://developers.google.com/identity/protocols/oauth2/scopes#oauth2.

37. GOOGLE. *Choose scopes for Google Drive API* [online]. 2024 [cit. 2025-03-01]. Dostupné z: https://developers.google.com/workspace/drive/api/guides/api-specific-auth.

38. GOOGLE CLOUD. *Steps to prepare for OAuth app verification* [online]. 2024 [cit. 2025-03-01]. Dostupné z: https://support.google.com/cloud/answer/13463073.

39. GOOGLE. *Using OAuth 2.0 for Web Server Applications – Incremental Authorization* [online]. 2026 [cit. 2026-03-02]. Dostupné z: https://developers.google.com/identity/protocols/oauth2/web-server#incrementalAuth.
40. GOOGLE. *Using OAuth 2.0 for Web Server Applications – Creating authorization credentials (Create Client)* [online]. 2026 [cit. 2026-03-02]. Dostupné z: https://developers.google.com/identity/protocols/oauth2/web-server#creatingclient.
41. GOOGLE CLOUD. *Manage App Audience – Publishing status: Testing* [online]. 2026 [cit. 2026-03-15]. Dostupné z: https://support.google.com/cloud/answer/15549945#publishing-status-testing.
42. GOOGLE. *Method: forms.get* [online]. 2025 [cit. 2026-03-26]. Dostupné z: https://developers.google.com/workspace/forms/api/reference/rest/v1/forms/get.
43. GOOGLE. *Resource: Form* [online]. 2026 [cit. 2026-03-26]. Dostupné z: https://developers.google.com/workspace/forms/api/reference/rest/v1/forms#Form.
44. GOOGLE. *Resource: FormResponse* [online]. 2026 [cit. 2026-03-26]. Dostupné z: https://developers.google.com/workspace/forms/api/reference/rest/v1/forms.responses#FormResponse.
45. DUDYCZ, Oskar. *PostgreSQL JSONB - Powerful Storage for Semi-Structured Data* [online]. 2025 [cit. 2026-03-26]. Dostupné z: https://www.architecture-weekly.com/p/postgresql-jsonb-powerful-storage.
46. TANSTACK. *TanStack Start – React Overview* [online]. 2026 [cit. 2026-03-17]. Dostupné z: https://tanstack.com/start/latest/docs/framework/react/overview.
47. TANSTACK. *TanStack Start – React Comparison* [online]. 2026 [cit. 2026-03-17]. Dostupné z: https://tanstack.com/start/latest/docs/framework/react/comparison.
48. GOOGLE. *Overview of the Places API Web Service* [online]. 2026 [cit. 2026-03-04]. Dostupné z: https://developers.google.com/maps/documentation/places/web-service/op-overview.
49. GOOGLE. *Google Maps Platform Pricing — Places Pricing* [online]. 2026 [cit. 2026-03-04]. Dostupné z: https://developers.google.com/maps/billing-and-pricing/pricing#places-pricing.
50. KOMOOT. *photon: an open source geocoder for OpenStreetMap data* [online]. 2026 [cit. 2026-03-04]. Dostupné z: https://github.com/komoot/photon.
51. TANSTACK. *Overview* [online]. 2026 [cit. 2026-03-17]. Dostupné z: https://tanstack.com/form/latest/docs/overview.

52. TANSTACK. *TanStack Query – Overview* [online]. 2026 [cit. 2026-03-17]. Dostupné z: https://tanstack.com/query/latest/docs/framework/react/overview.

53. TANSTACK. *TanStack Start – Routing Guide* [online]. 2026 [cit. 2026-03-17]. Dostupné z: https://tanstack.com/start/latest/docs/framework/react/guide/routing.

54. STEPHANIE SUSNJARA, Ian Smalley. *What is software testing?* [Online] [cit. 2026-03-19]. Dostupné z: https://www.ibm.com/think/topics/software-testing.

55. ERAN KINSBRUNER. *Different Types of Testing in Software* [online]. 2026 [cit. 2026-03-19]. Dostupné z: https://www.perfecto.io/resources/types-of-testing.

56. ZAIDI, Sohaib. *Integration Testing: A Comprehensive Guide with Best Practices* [online]. 2024 [cit. 2026-03-19]. Dostupné z: https://www.opkey.com/blog/integration-testing-a-comprehensive-guide-with-best-practices.

57. VITEST. *Vitest Documentation* [online]. 2026 [cit. 2026-03-26]. Dostupné z: https://vitest.dev/.

58. JUNIT TEAM. *JUnit* [online]. 2026 [cit. 2026-03-20]. Dostupné z: https://junit.org/.

Obsah příloh

```
/
├── readme.txt ..... stručný popis obsahu média
├── exe ..... adresář se spustitelnou formou implementace
├── src
│   ├── impl ..... zdrojové kódy implementace
│   └── thesis ..... zdrojová forma práce ve formátu LaTeX
└── text ..... text práce
    └── thesis.pdf ..... text práce ve formátu PDF
```